2026-07-04

MicroPython Tutorial

Table of Contents

1

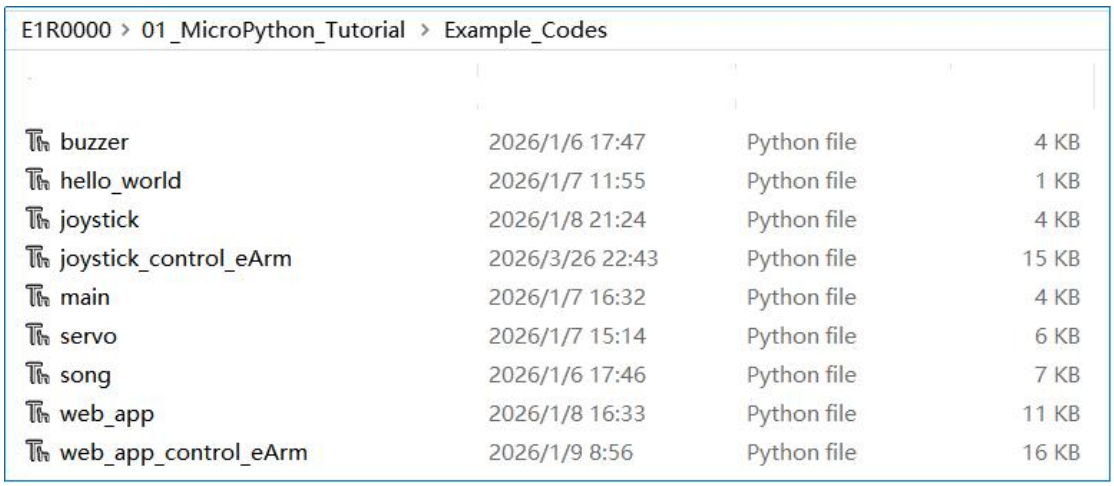
Programming Learning

1.1 Before You Begin: Important Notes

The ESP32-C3 control board is preloaded with firmware that allows the robotic arm to operate as described in the "**Assembly and User Guide**". This firmware will remain on the board until you flash the MicroPython firmware using the Thonny IDE.

Note: Flashing MicroPython firmware is completely reversible. After installing MicroPython on your ESP32-C3 board, you can switch back to Arduino at any time by simply uploading an Arduino sketch using the Arduino IDE.

The tutorial package includes a collection of MicroPython example programs. Before continuing, please download the tutorial package and remember the folder where it is saved, as you will need to open these example files later.



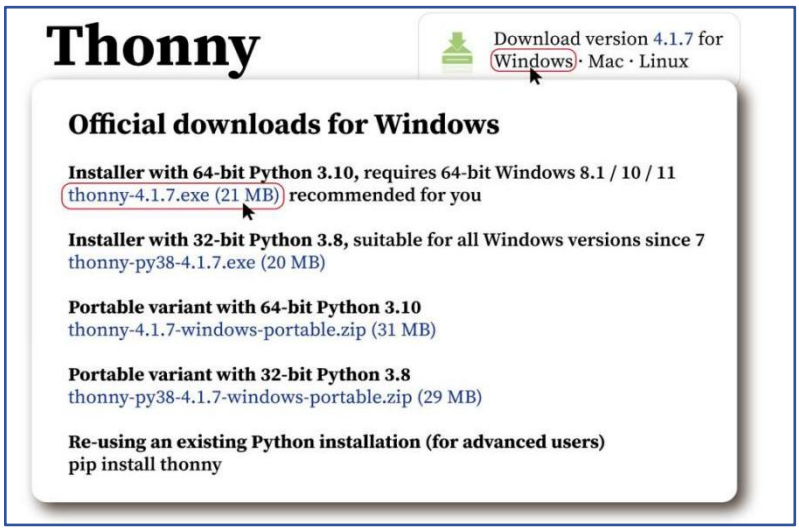
1.2 Install Thonny IDE

Official website of Thonny: https://Thonny.org

Thonny supports three mainstream operating systems: Windows, macOS, and Linux. Follow the section for the operating system you're using.

Operating System	Installation Methods
Windows	Ⅰ) Installing Thonny IDE – Windows PC
Mac OS	Ⅱ) Installing Thonny IDE – Mac OS

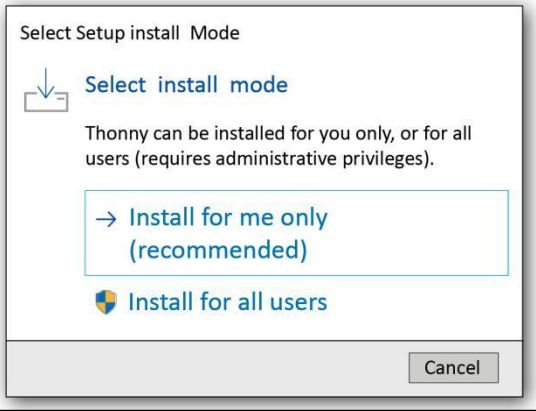
I) Installing Thonny IDE – Windows PC



1. Download the installer Thonny-4.1.7.exe, double-click to run it.

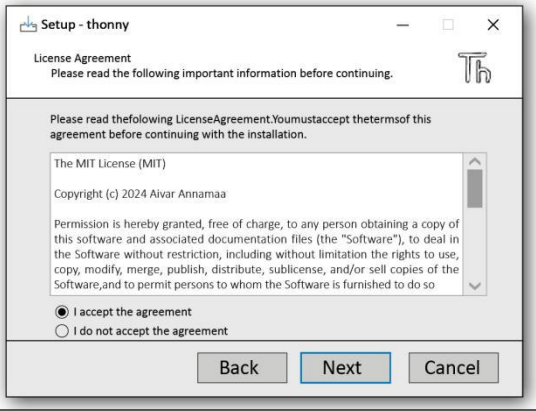
2. Keep clicking "Next" to complete the installation.

1. Click Install for me only

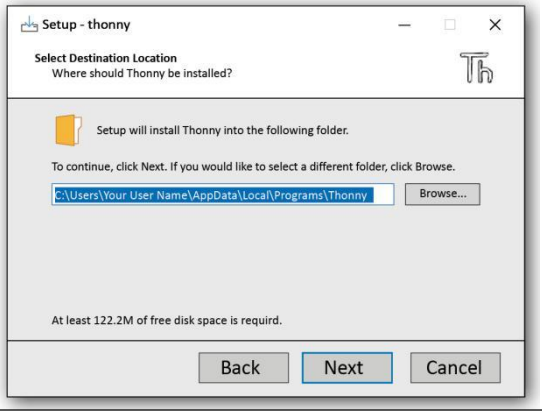


2. Click Next

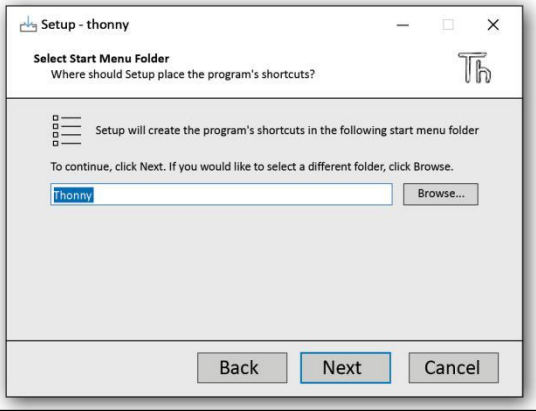
3. Click Next



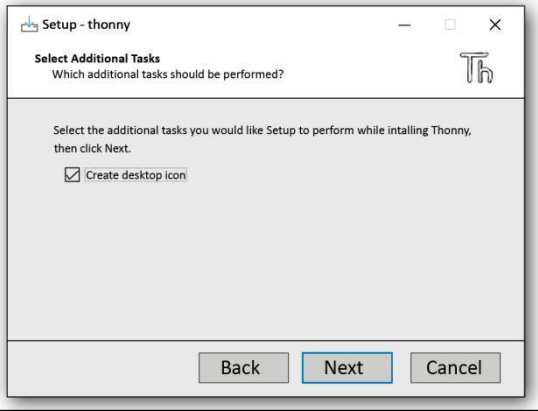
4. Click Next or click "Browse" to modify the default installation path.



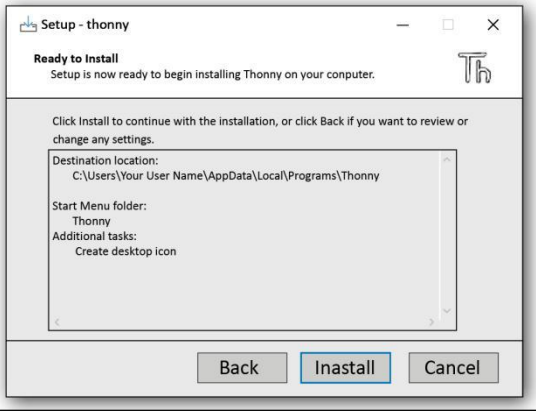
5. Click Next



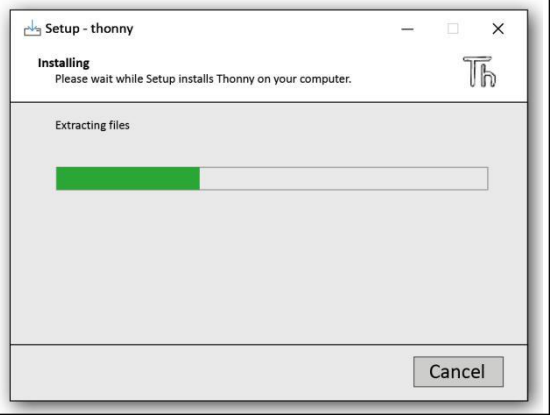
6. Check Create desktop icon, click Next



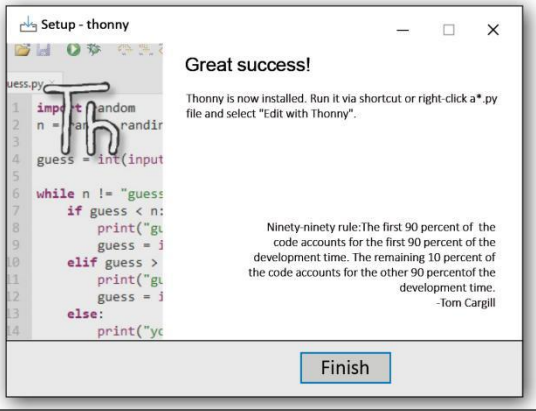
7. Click Install



8. Wait a few seconds for the installation to complete.



9. Finish



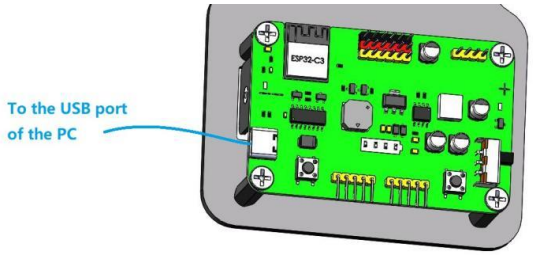
3. After installation, find the desktop application icon to run it.

1.3 Install CH340 USB driver

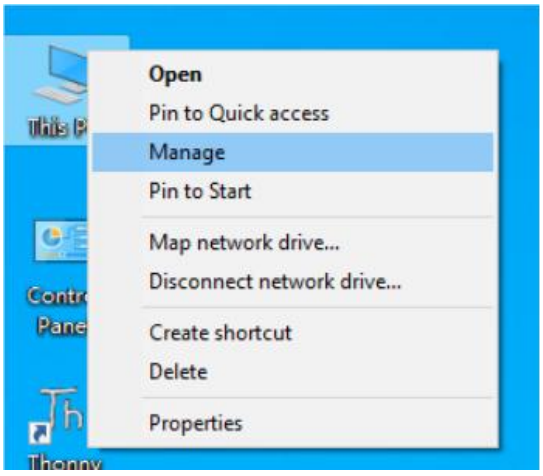
Operating System	Installation Methods
Windows	Ⅰ) **Install the Driver on Windows system**
Linux	Ⅱ) Install the Driver on Linux system
Mac OS	Ⅲ) Install the Driver on MAC system

Ⅰ) Install the Driver on Windows system

1. Connect your ESP32 board to your computer using a USB cable.

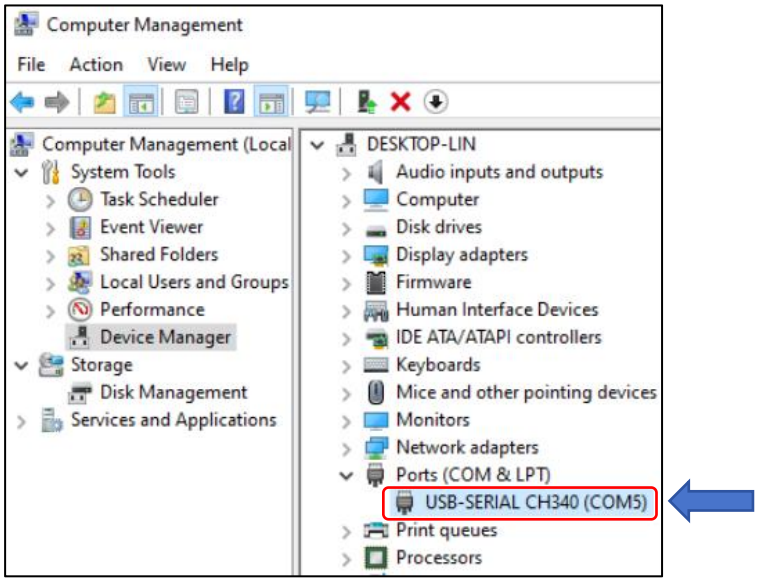


2. Go to the desktop of your computer, select “This PC” and right-click to select “Manage”



3.Check whether the CH340 has been installed

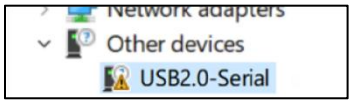
If the driver is properly installed, when the ESP32 board is connected to the computer via a USB cable, the device "USB-Serial CH340 (COMX)" can be found under the path "Device Manager" → Ports (COM & LPT), as shown in the figure below.



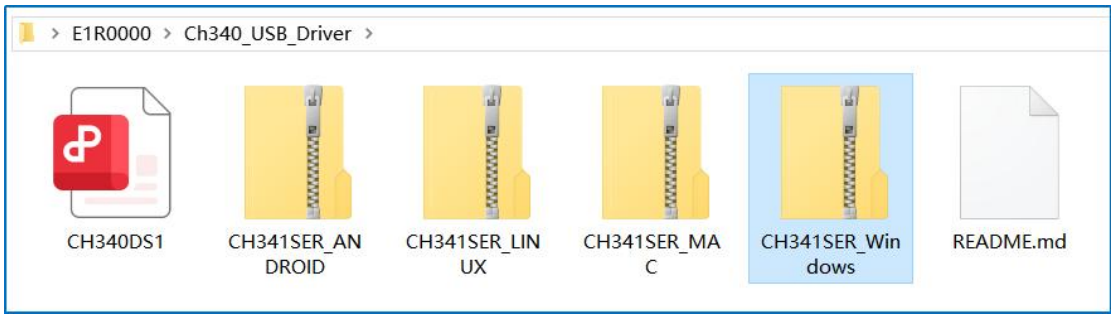
Note: The displayed driver name is “USB-SERIAL CH340 (COMx)”, “x” can be any number, it depends on what number your computer assigns to the ESP32 device. If the driver is already installed, skip the driver installation step and proceed to the next chapter.

4.Install CH340 driver manually

If the CH340 driver is not installed, the Windows Device Manager typically shows a "USB2.0-Serial" or "Unknown Device" with a yellow exclamation mark under "Ports (COM & LPT)" or "Other Devices". This indicates that the computer has detected the hardware but cannot use it properly. As a result, a COM port number will not be assigned, preventing serial communication.

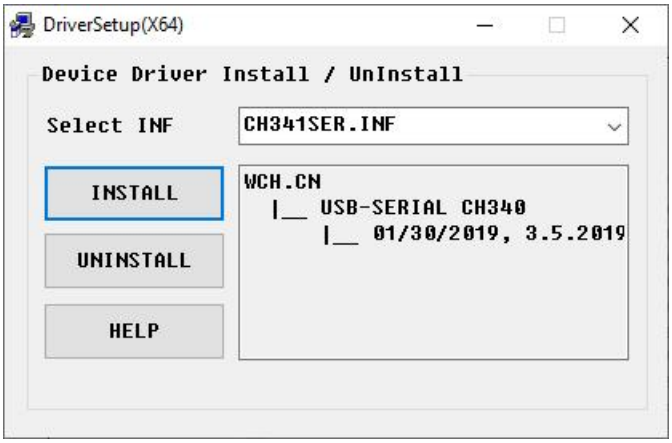


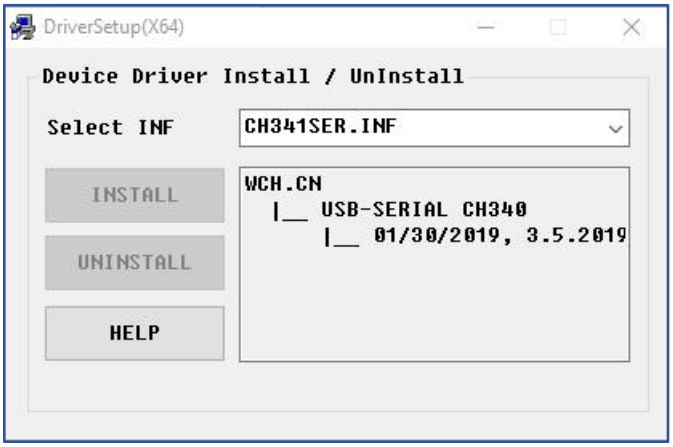
Driver file is provided in the tutorial package:



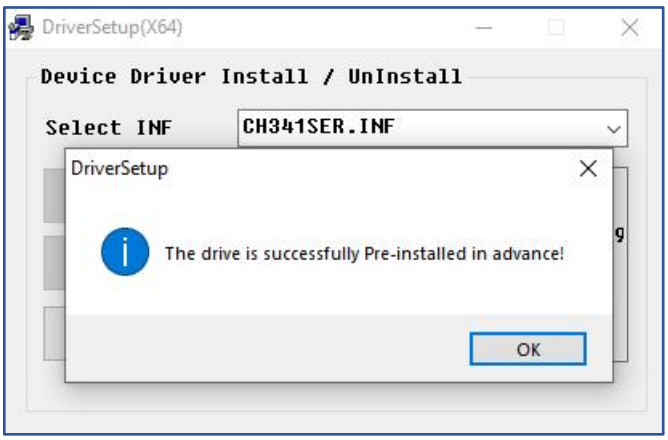
Unzip the file we provided and double-click **"SETUP"** to run the installer:

Click "INSTALL" and wait for the installation to complete.





Install successfully. Close all interfaces.

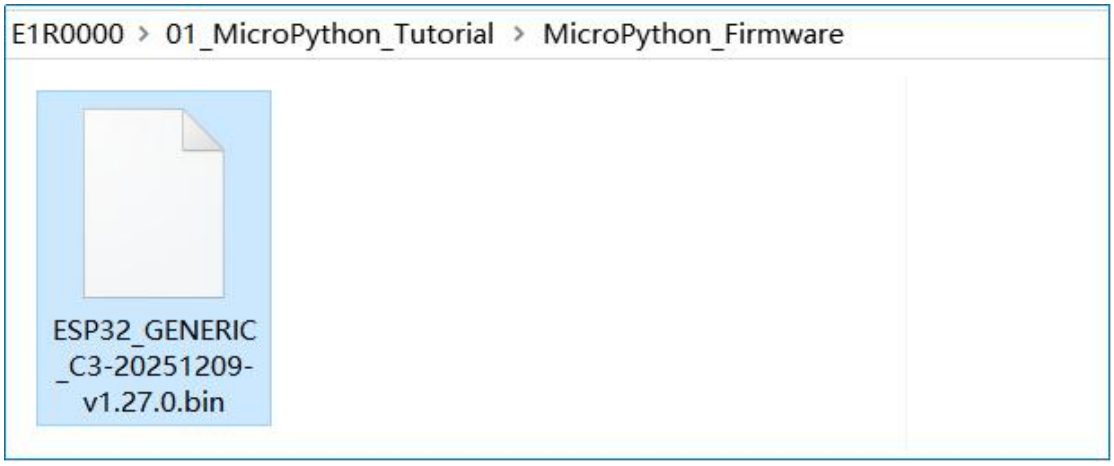


1.4 Flashing MicroPython Firmware using Thonny IDE

To upload code to an ESP32 from Thonny IDE, you must first ensure the ESP32 is flashed with MicroPython firmware and Thonny is configured to use the MicroPython interpreter.

Firmware used in this tutorial is ESP32_GENERIC_C3-20251209-v1.27.0.bin

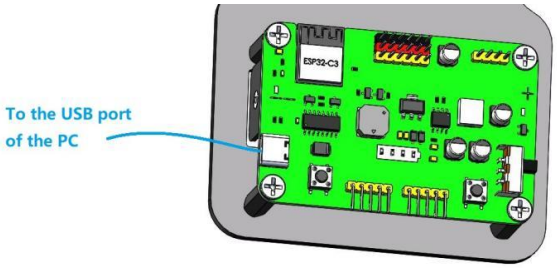
This MicroPython Firmware file is provided in the tutorial package:



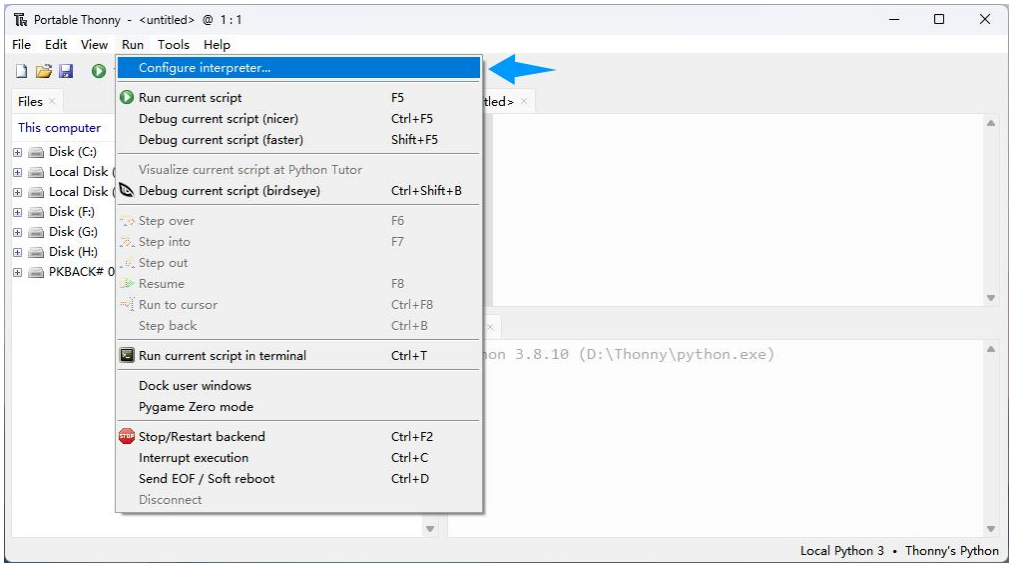
Or download it from the official website
List of MicroPython firmware for ESP32-C3:
https://micropython.org/download/ESP32_GENERIC_C3/

Follow the next steps:

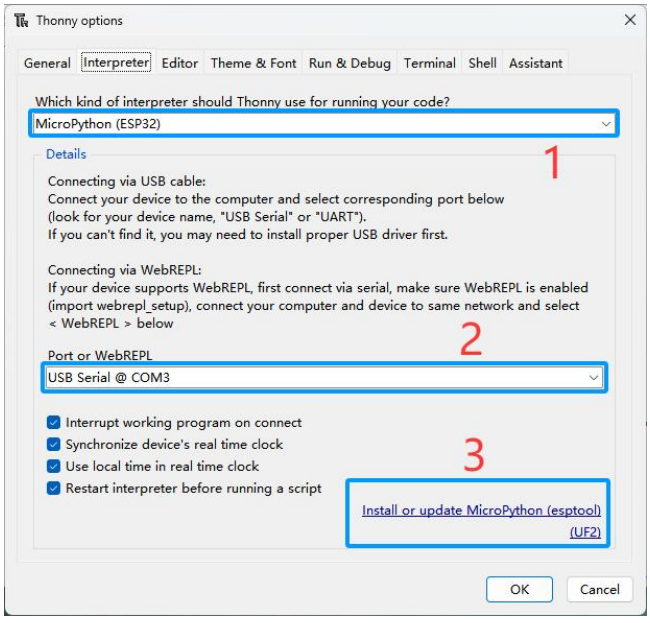
1) Connect your ESP32 board to your computer.



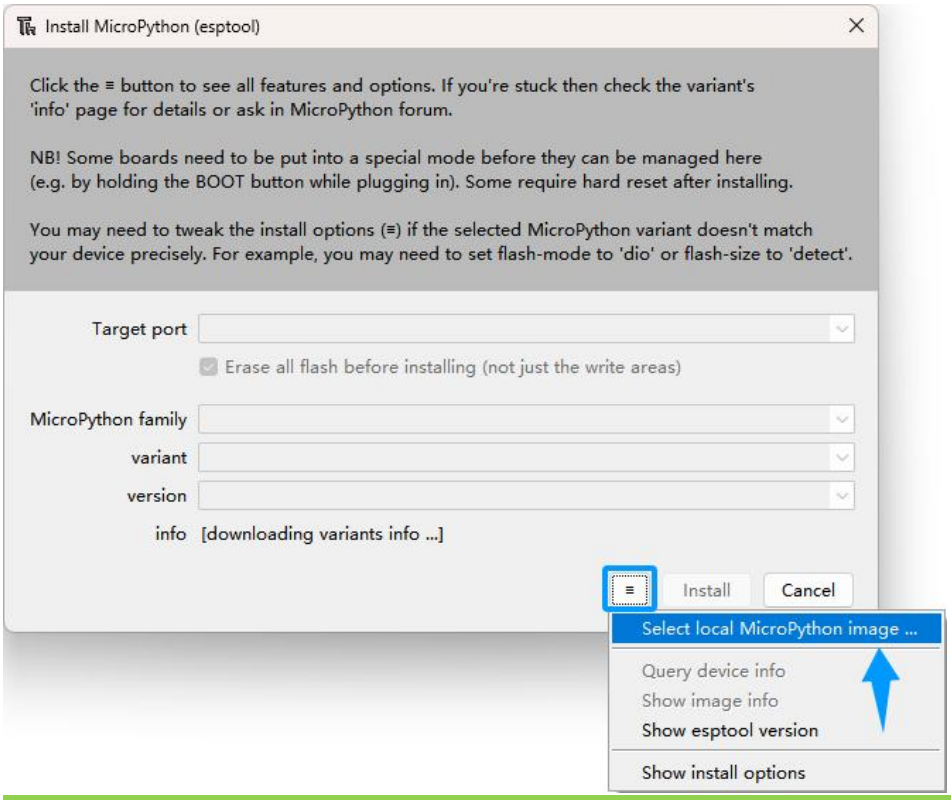
2) Open Thonny IDE. Click “Run” and select “Configure interpreter...”



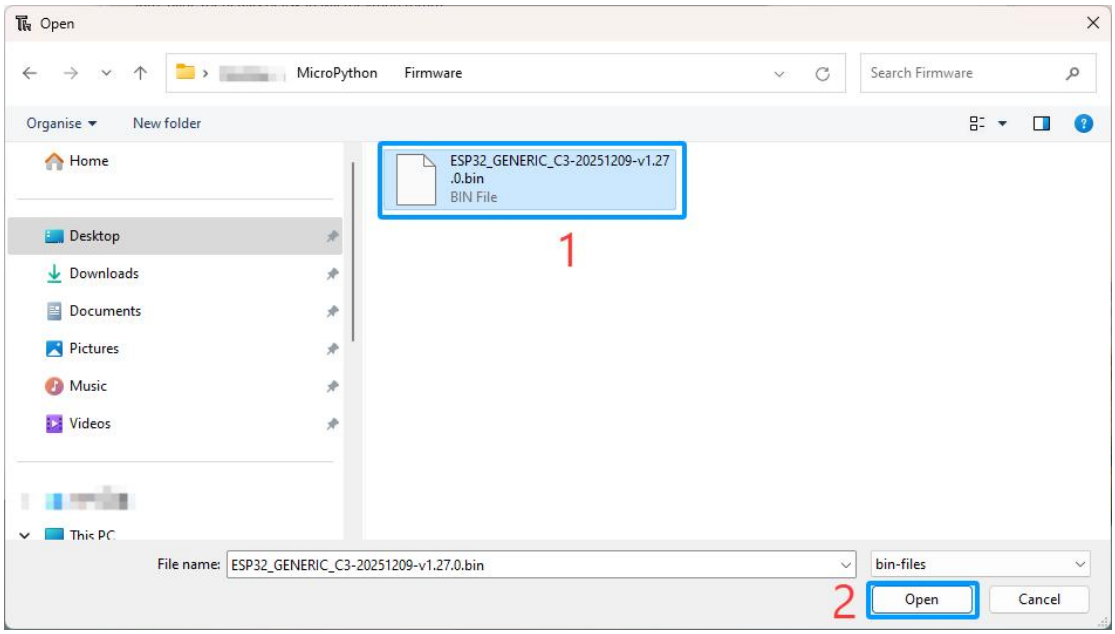
3) Select “Micropython (ESP32)”, select “USB Serial @ COMX(Depending on the number assigned by the Device Manager), and then click the long button under “Install or update MicroPython(esptool)(UF2)”.



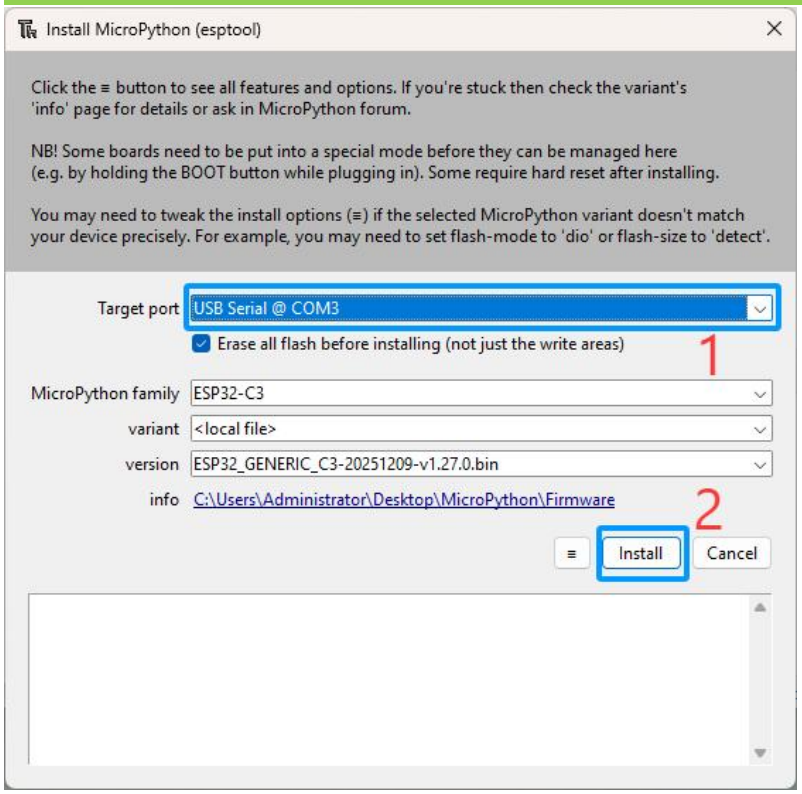
4) The following dialog box pops up. Select “Select local MicroPython image...”.



5) Select the microPython firmware “ESP32_GENERIC_C3-20251209-v1.27.0.bin” provided in the tutorial package. Click "Open".



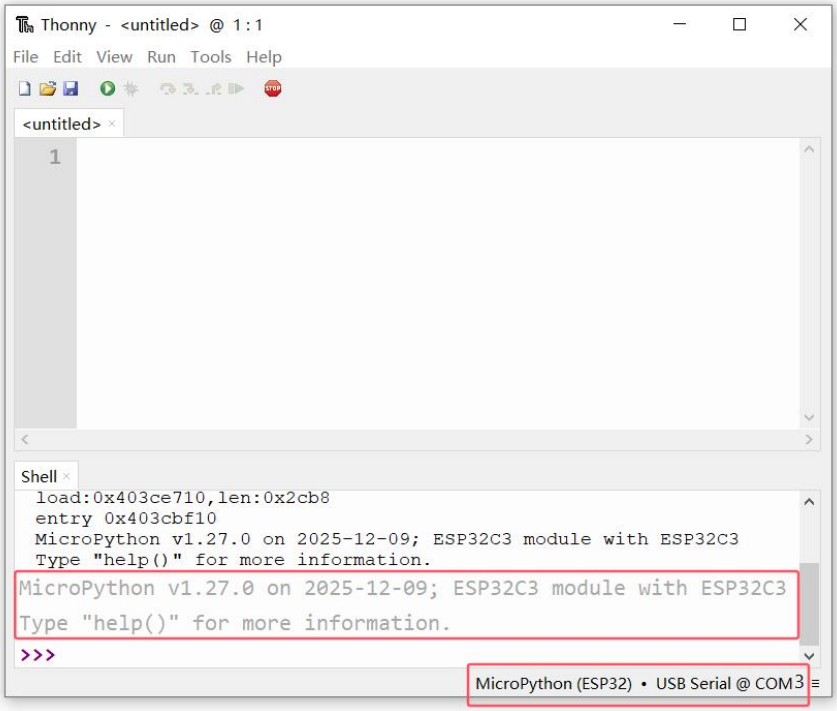
6) Select “USB Serial @ (COM3)”, and then click “Install”.



Wait for the installation to be done, and then click "Close".

7) Testing the Installation

When the installation is completed, you can close the window. You should get the following message on the Shell (see the picture below), and at the bottom right corner, it should have the Interpreter it’ s using and the COM port.



The REPL (Shell)

You should also see the **>>> symbol**
This symbol represents the MicroPython REPL (Read-Eval-Print-Loop) prompt. This symbol means you ' re interacting with MicroPython (the board with MicroPython firmware installed) in interactive mode in real time. This simply means that you can enter and execute commands directly.

Type **print('Hello')** in the Shell after the prompt >>> and press Enter
It should print Hello on the Shell right after you enter the command.

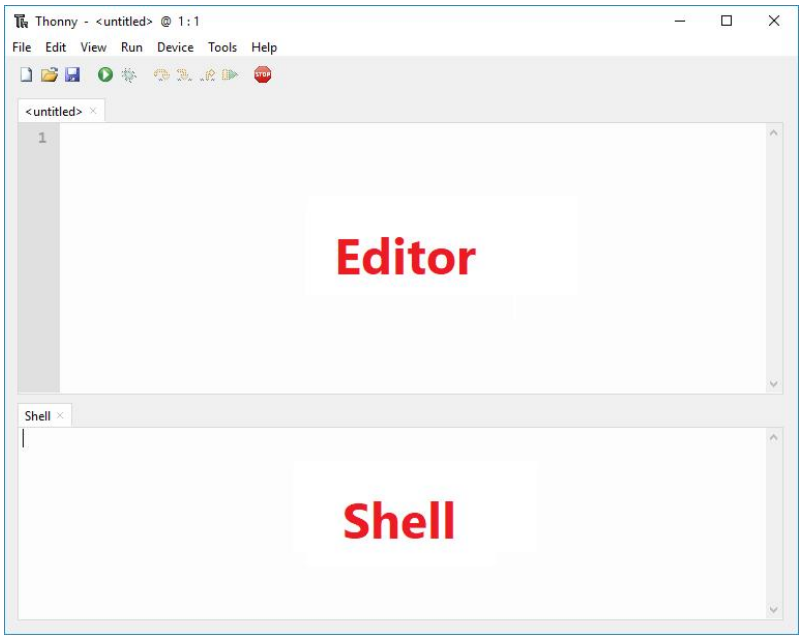
```
>>> print('Hello')
  Hello
```

If everything is working as expected, it means you have successfully installed Thonny IDE, as well as MicroPython firmware on your ESP32 board.

1.5 Thonny IDE Overview

Open Thonny IDE. There are two different sections:

the Editor and the MicroPython Shell/Terminal:



The Editor section is where you write your code and edit your .py files. You can open more than one file, and the Editor will open a new tab for each file.

In the MicroPython Shell, you can type commands to be executed immediately by your ESP32 board without the need to upload new files. The terminal also provides information about the state of an executing program, shows errors related to the uploading process, syntax errors, prints messages, etc…

The Icons

In the Thonny IDE's top section, there are some menu bars that are frequently used during programming:

 : Create a new python file.

 : The open folder icon allows you to open an existing file from your computer. This might be useful if you come back to a program that you worked on previously.

 : The floppy disk icon allows you to save your code. To prevent loss of the code you are writing, it is recommended to develop the habit of frequently clicking this menu.

 : Run the python code of the current page (being viewed) of the **Editor**.

: Stop the function being debugged.

The following features are currently not supported by the MicroPython firmware for ESP32 C3 and can be understood briefly:

 : The bug icon allows you to debug your code.

 : The arrow tells Python to take a big step, meaning jumping to the next line or block of code.

 : The arrow tells Python to take a small step, meaning diving deep into each component of an expression.

 : The arrow tells Python to exit out of the debugger.

 : Exit the debug mode.

1.6 Running Your First Code - Run current code in Thonny

1.Ensure that you have configured the development environment.

At this point, you should have:

- ✓ Install a sufficiently charged 18650 battery and turn on the power switch.
- ✓ Connected the ESP32 board to the computer.
- ✓ Ch340 driver installed on your computer
- ✓ Thonny IDE installed on your computer
- ✓ ESP32 board flashed with MicroPython firmware

To get you familiar with the process of writing a file and executing code on your ESP32 board, we'll upload a new script that simply controls the built-in buzzer of the ESP32 to produce sound.

2. Running the Code

When you open Thonny IDE for the first time, the Editor shows an untitled file. Copy the following script to that file:

```
from machine import Pin, PWM
import time

buzzer = PWM(Pin(9))  # Initialize buzzer on GPIO pin 9 as a PWM output
while True:
    buzzer.freq(1000) # Set frequency to 1000 Hz (1 kHz tone)
    buzzer.duty(512)  # Set duty cycle to 50% (512/1024) to produce sound
    time.sleep_ms(200)# Keep the buzzer active for 200 milliseconds
    buzzer.duty(0)    # Turn off the buzzer by setting duty cycle to 0%
    time.sleep(1)     # Wait for 1 second before repeating the beep
```

IMPORTANT: How to keep the correct format

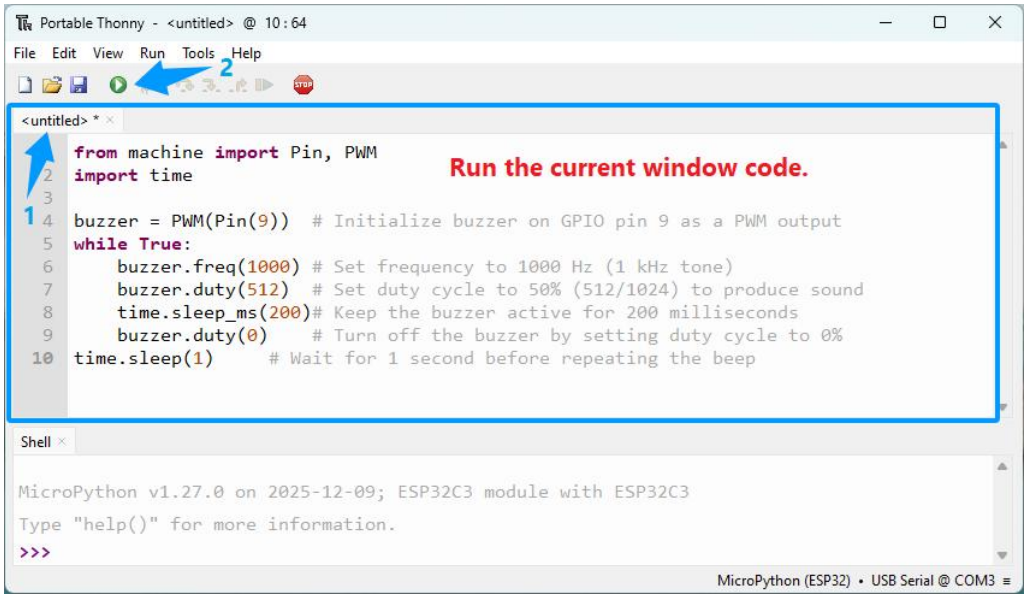
<untitled> *

```
1  from machine import Pin, PWM
2  import time
3
4  buzzer = PWM(Pin(9)) # Initialize buzzer on GPIO pin 9 as a PWM output
5  while True:
6      buzzer.freq(1000) # Set frequency to 1000 Hz (1 kHz tone)
7      buzzer.duty(512) # Set duty cycle to 50% (512/1024) to produce sound
8      time.sleep_ms(200)# Keep the buzzer active for 200 milliseconds
9      buzzer.duty(0) # Turn off the buzzer by setting duty cycle to 0%
10     time.sleep(1) # Wait for 1 second before repeating the beep
```

A blank line

Always use 4 spaces for indentation in Python

When running the code for the first time: after you see the ' **>>>** ' prompt, simply click the green '**Run**' button in the Thonny IDE.



Ensure to turn on the power switch of the control board, the buzzer will then emit sound.

To stop the execution of the program, you can hit the STOP button or simply press CTRL+C.

Note:

Just running the file with Thonny doesn't copy it permanently to the ESP32 filesystem. This means that if you unplug it from your computer and apply power to the board, nothing will happen because it doesn't have any MicroPython file saved on its filesystem.

The Thonny IDE Run function is useful to test the code, but if you want to upload it permanently to your board, you need to create and save a file to the board's filesystem. We'll cover this next.

1.7 Uploading Code to the ESP32 Board - Run code in ESP32

Method 1: Save the Code as main.py (to ESP32 Device)

1) After running the buzzer code in the previous section (in run/current mode), stop the execution by clicking the red Stop icon.

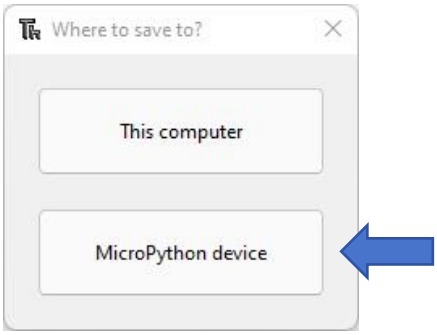
Now you can save the code either to your computer or directly upload it to the ESP32 MicroPython device:

Click the Save icon, or go to File > Save As…

2) In the save dialog, choose where to save the file:

Select This computer if you want to save it locally on your PC.

Here we select MicroPython device (ESP32) to upload it directly to the ESP32 board.

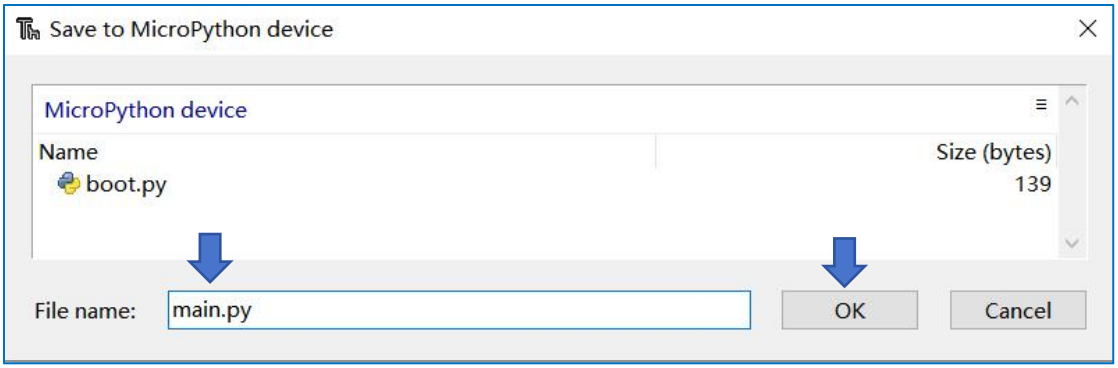


Note:

Q：The Thonny file panel doesn't show "MicroPython Device"?

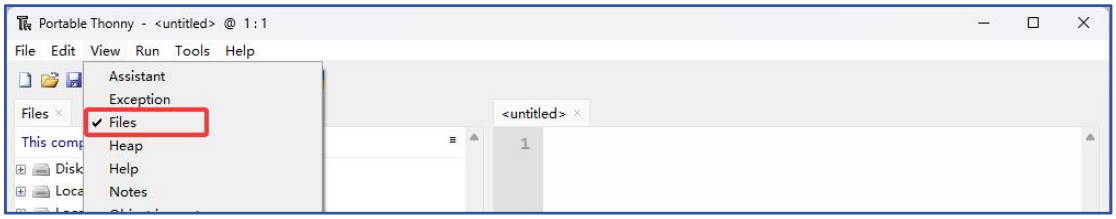
A：Every time the USB port of the computer is connected to the control board, this phenomenon occurs because Thonny IDE does not have the function of automatically detecting the connection of ESP32 devices. Click the **STOP** button or reselect "**MicroPython (ESP32) ▪USB Serial @ COMx**" at the lower right corner of Thonny to solve this problem.

3) Name the file **main.py** and click OK. Note: Saving the file as main.py on the ESP32 device allows the board to run this code automatically every time it powers on or restarts.

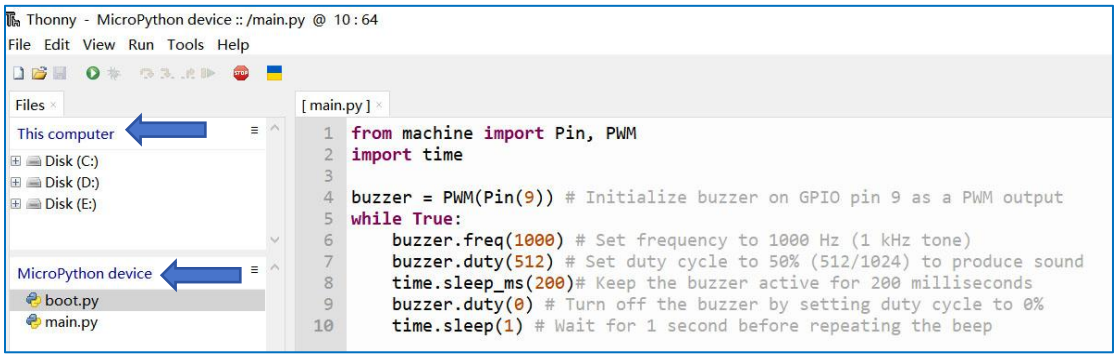


Notice that the window that pops up shows the files currently saved on the board's filesystem. There should be a file called **boot.py.** That file is created by default when you burn MicroPython firmware.

4) Click "View" in the menu bar and select "Files" from the dropdown.



5) You will see two file panels: the a panel labeled "This computer" (your local files) and the panel labeled "MicroPython device" (the connected ESP32's internal storage).



6) After saving the file, you can remove and apply power again to the board. Notice that the buzzer on the ESP32 board will automatically emit a sound when you apply power to it. This means the main.py file was successfully uploaded to the **MicroPython device** and that it is running that file automatically.

Important: when you name a file main.py, the ESP32 will run that file automatically on boot (when it starts). If you call it a different name, it will still be saved on the board filesystem, but it will not run automatically on boot.

Can You Ignore boot.py?

Yes, you can safely ignore the boot.py file. But there is no need to delete it. For almost all beginner and most intermediate ESP32 MicroPython projects (e.g., LED blinking, sensor data reading, basic WiFi connection, simple serial communication). The ESP32's MicroPython runtime operates fully normally, and all your project logic can be directly written in main.py (including simple initialization steps like pin or sensor setup). You only need to use boot.py when you need to separate system-level initialization from main application logic for cleaner code organization.

In short: boot.py prepares the "system environment", main.py runs the "actual project", and REPL is the "debug tool" — the three work together to form the complete running logic of ESP32 MicroPython, with boot.py being an optional but powerful supplement for system-level configuration.

Method 2: main.py calls other files

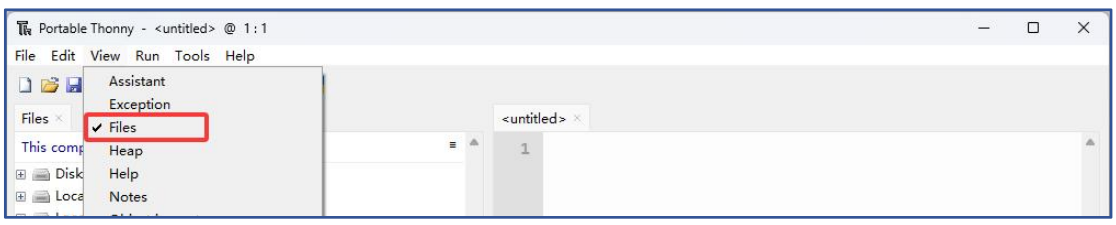
Comparison of the Two Methods

Method	Operation Process	Advantages	Disadvantages	Applicable Scenarios
① Save as main.py (direct upload)	1. Open the tutorial code	All the code is written in the main.py file, which is suitable for simple single-file projects.	1. High coupling, inconvenient for debugging	Single-file experiments, one-time testing
	2. Click ***Save as*** → Select device		2. Requires re-uploading entire code for modifications	
	3. Name it main.py		3. Not modular	
② main.py calls other files	1. Upload functional code (e.g., buzzer.py) to the device	1. Modular, easy to maintain	Requires slightly more file management steps	Multi-module projects, long-term development
	2. Create a new main.py file and write the code to call other files in main.py.	2. Allows independent debugging of submodules		
	3. Upload main.py to the ESP32 device.	3. Supports code reuse		

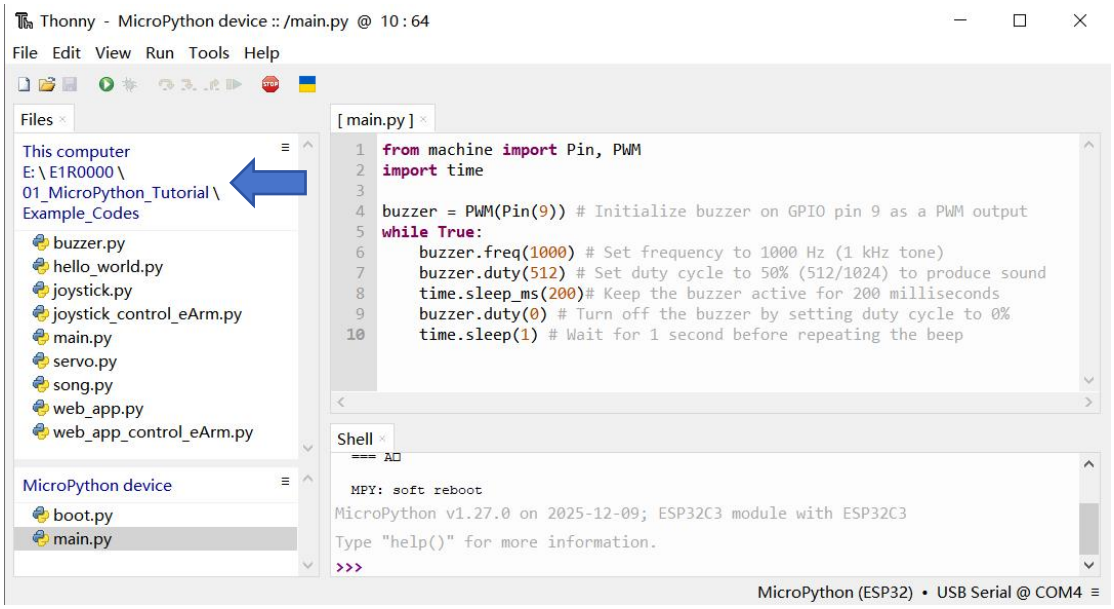
It is recommended to choose the **second** method as it is more professional, flexible and in line with the recommended practices of embedded development.

We provide the **xxx.py** files and **main.py** with different functions for your use in the tutorial package. The code in main.py will call any the **xxx.py** files under "**MicroPython device**" except main.py and boot.py.

1) Click "View" in the menu bar and select "Files" from the dropdown.



2) Click “ This computer ” , navigate to the following folder where you saved the tutorial package:E1R0000\01_MicroPython_Tutorial\Example_Codes
to access the MicroPython code examples provided in the tutorial.

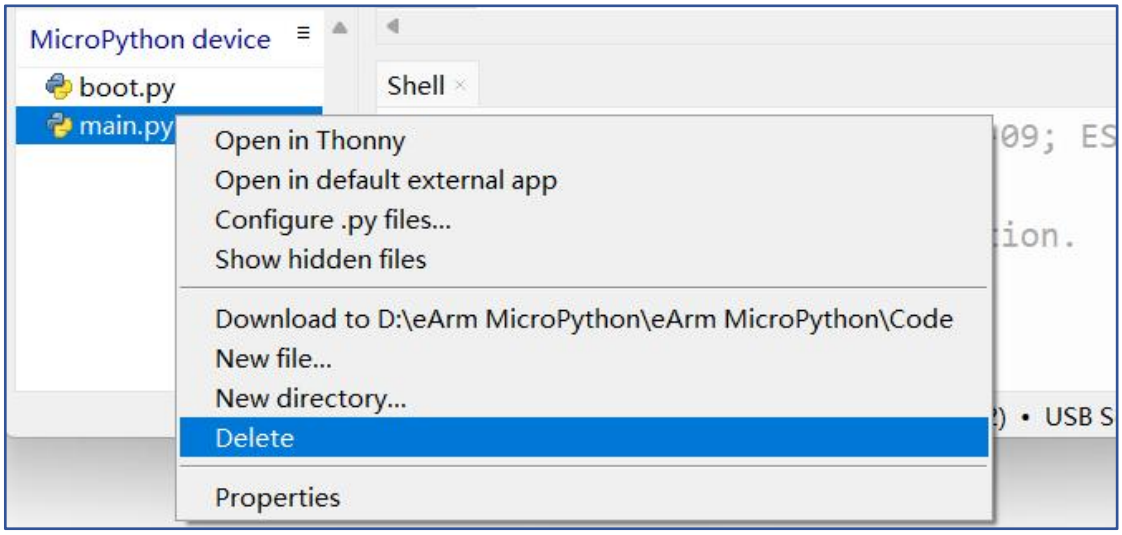


Note:

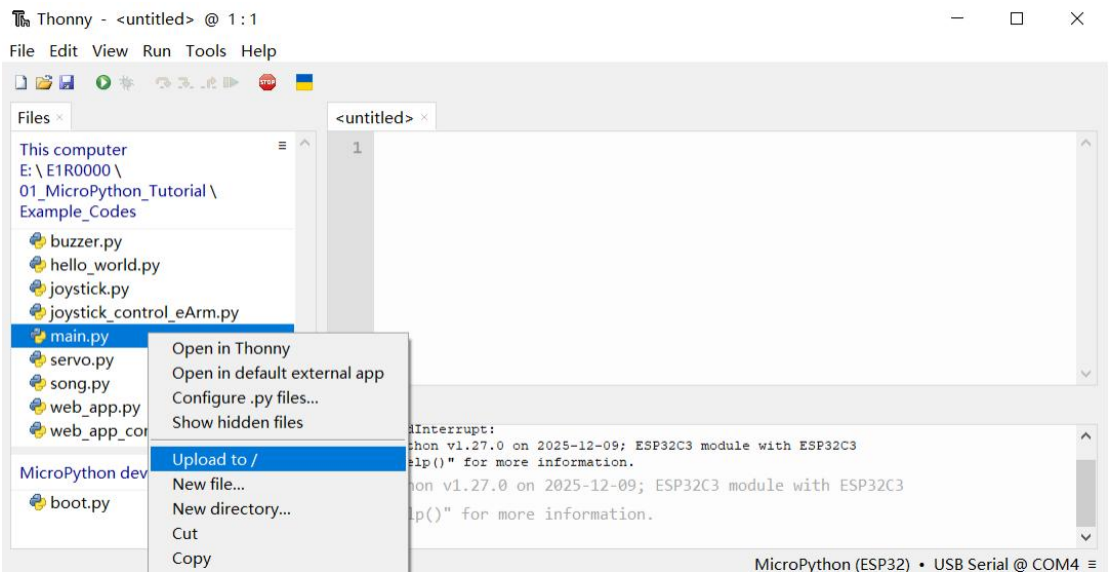
Q：The Thonny file panel doesn't shows "MicroPython Device" ?

A：Every time the USB port of the computer is connected to the control board, this phenomenon occurs because Thonny does not have the function of automatically detecting the connection of ESP32 devices. Click the **STOP** button or reselect "**MicroPython (ESP32) •USB Serial @ COMx**" at the lower right corner of thony to solve this problem.

3) Right-click “main.py” previously uploaded to make the buzzer sound in “MicroPython device”, Delete it from ESP32-C3’ s root directory.

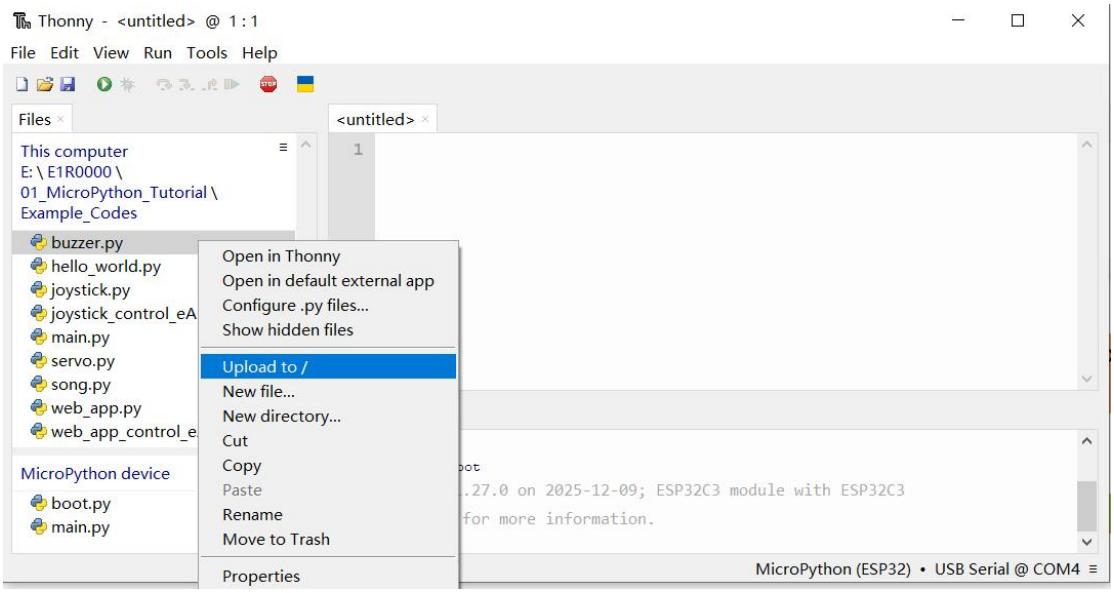


4) Right-click “main.py” in your local folder(This computer), and click "Upload to/"
This is the main program we provided. Its purpose is to call and run the other MicroPython example files.

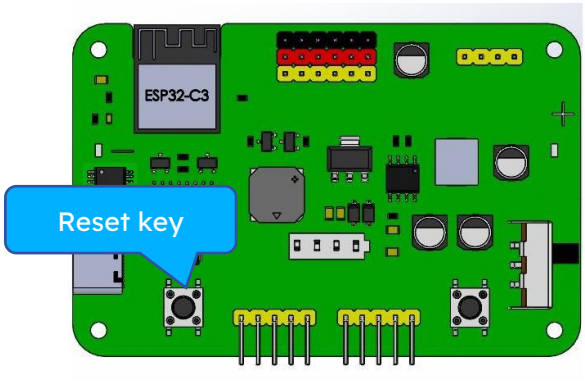


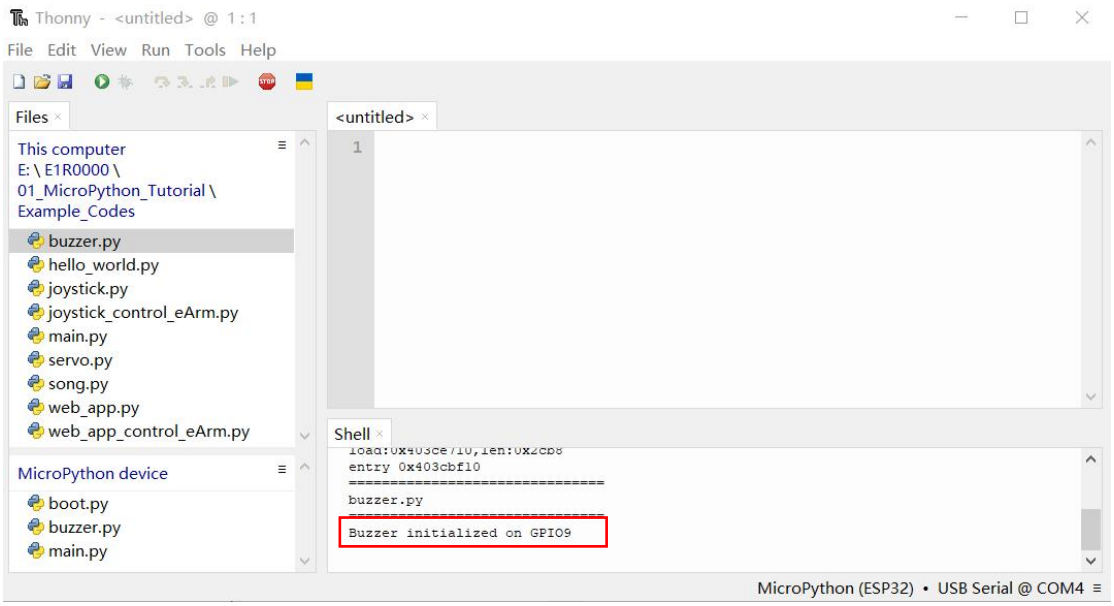
Important: You must stop running online before uploading files to "MicroPython device"! (Click the "**STOP**" menu in Thonny)

5) Right-click “buzzer.py” in your local folder, and click "Upload to/"

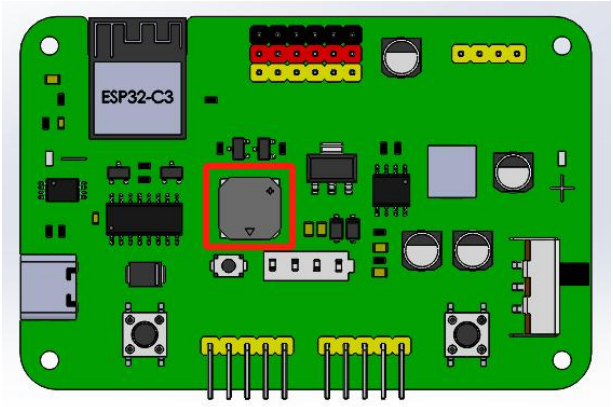


6) Press the reset key of the esp32 board, you will see the code is executed.





7) The buzzer on the ESP32-C3 board will keep making sounds when turn on the power switch.

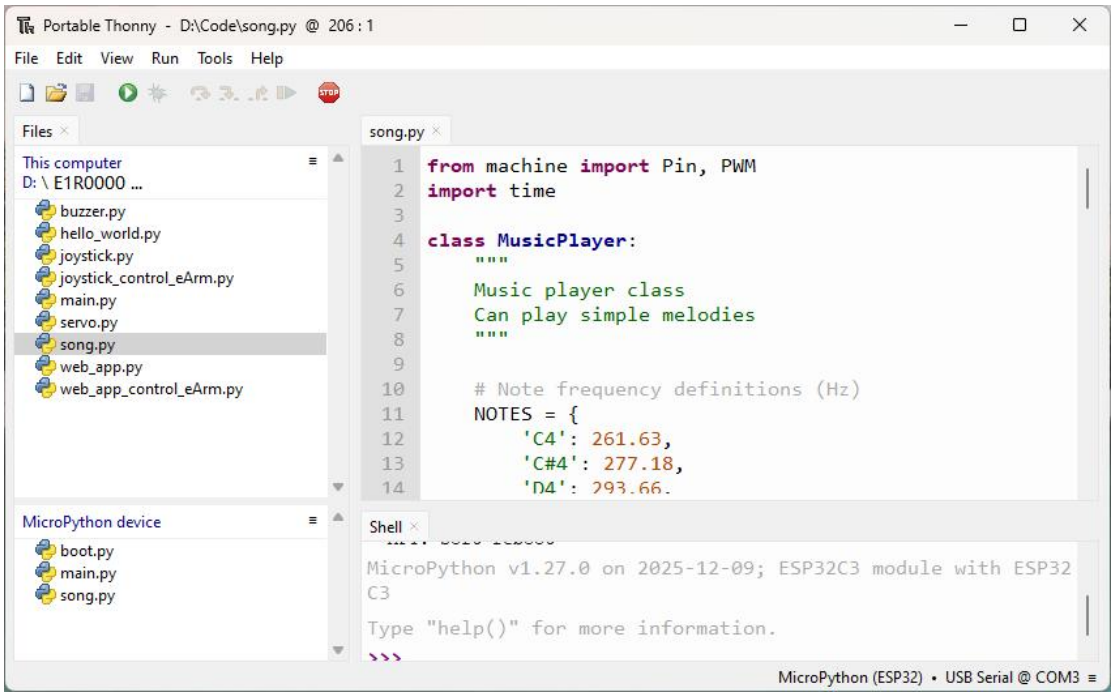


Important: The shared main.py file we provided is designed to call only one of the non - main.py code files at a time. If you upload multiple such files together, pressing reset may cause the system to behave erratically — it might randomly execute a different file instead of the intended one.

1.8 Code Examples

1.8.1 Play Songs

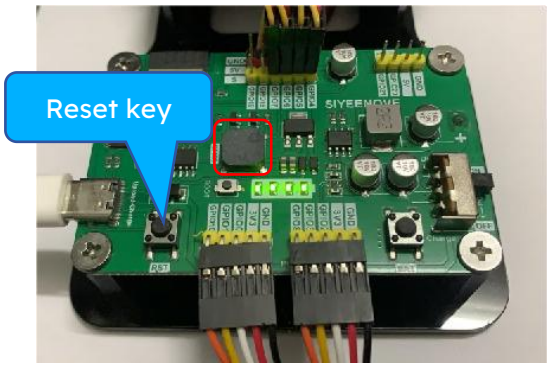
1	Delete the previously uploaded **buzzer.py** from the MicroPython device, because the provided main.py is designed to call only one module file.
2	If any code is running, please click the **Stop** button.Then right-click on **song.py** and select "**Upload to /**"
3	The **song.py** file will appear in the MicroPython device file list.
4	Ensure the control board's power switch is turned ON and the battery has sufficient charge.
5	Press the **Reset** button on the esp32 control board to restart it. The board's buzzer will then start playing music.
6	To **view** the code: Navigate to the **song.py**, then double-click the file to open it in Thonny.



Note:

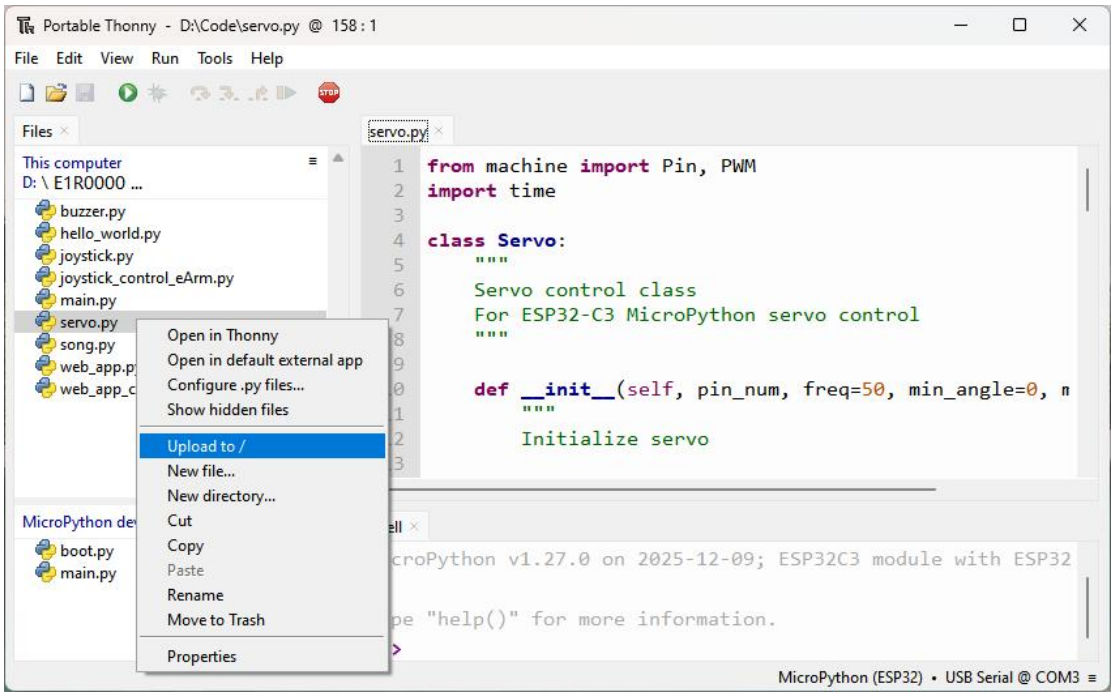
Q：The Thonny file panel doesn't shows "MicroPython Device" ?

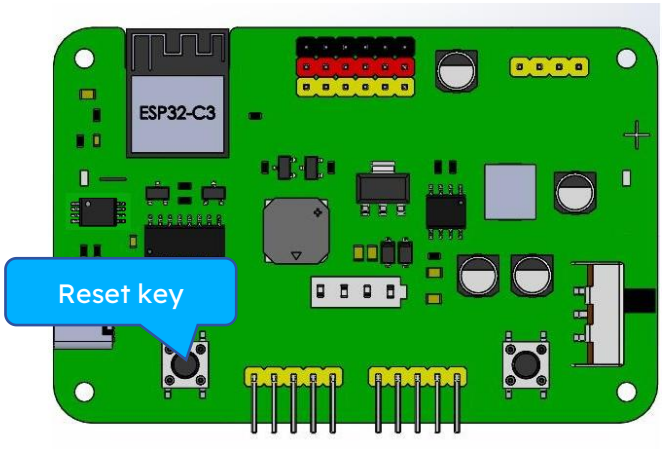
A：Every time the USB port of the computer is connected to the control board, this phenomenon occurs because Thonny does not have the function of automatically detecting the connection of ESP32 devices. Click the **STOP** button or reselect "**MicroPython (ESP32) ▪USB Serial @ COMx**" at the lower right corner of thony to solve this problem.



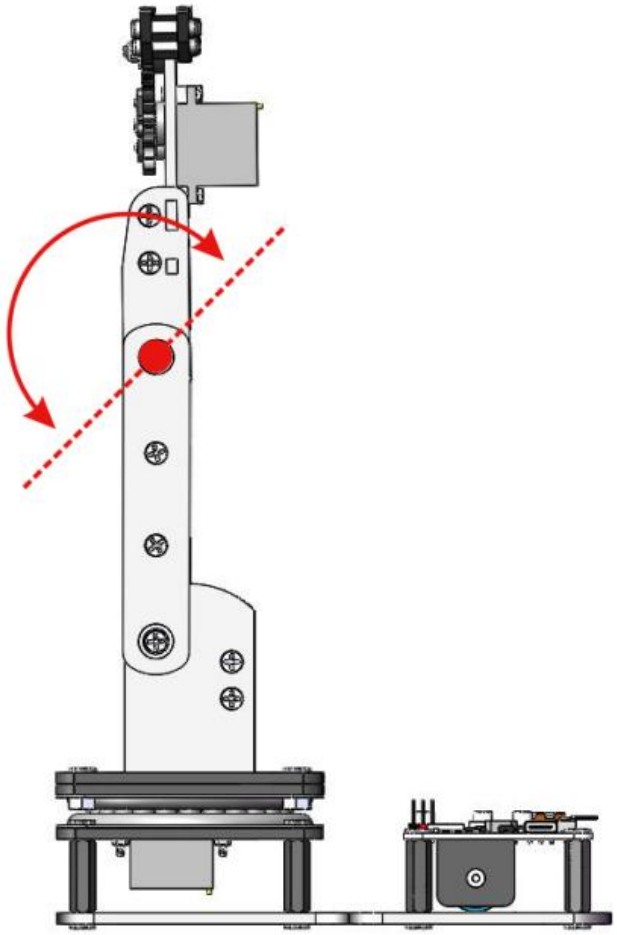
1.8.2 To Drive a Servo

1	Delete the previously uploaded **song.py** from the MicroPython device, because the provided main.py is designed to call only one module file.
2	If any code is running, please click the **Stop** button.Then right-click on **servo.py** and select "**Upload to /**"
3	The **servo.py** file will appear in the MicroPython device file list.
4	Ensure the control board's power switch is turned ON and the battery has sufficient charge.
5	Press the **Reset** button on the esp32 control board to restart it. The eArm's servo C will swing 0-180, 180-0 in a cycle:
6	To **view** the code: Navigate to the **servo.py**, then double-click the file to open it in Thonny.



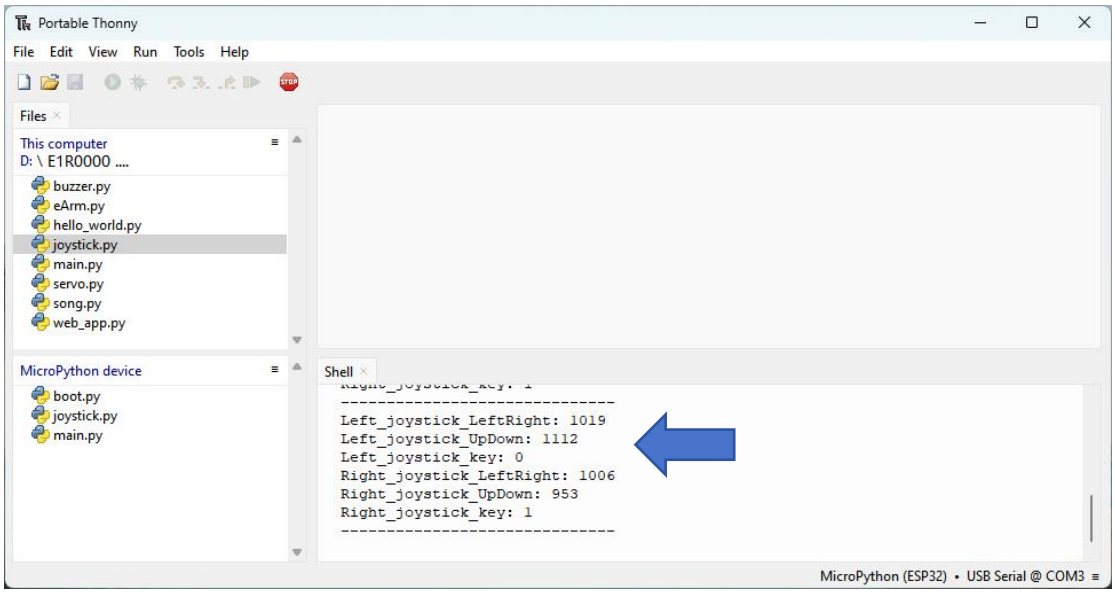


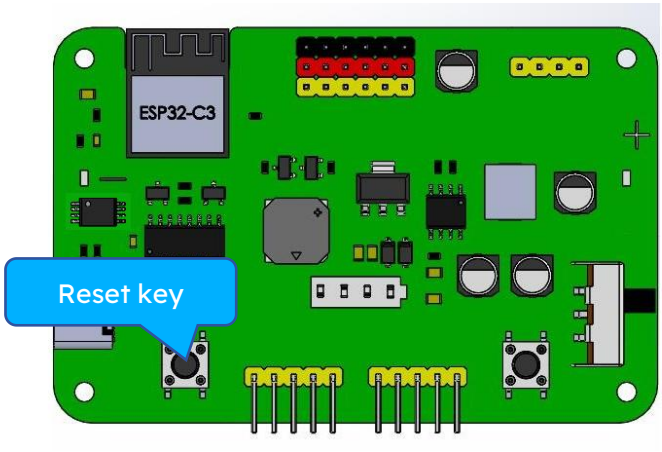
After the code is uploaded successfully, the eArm's servo C will swing 0-180, 180-0 in a cycle:



1.8.3 Read the Value of the Joystick

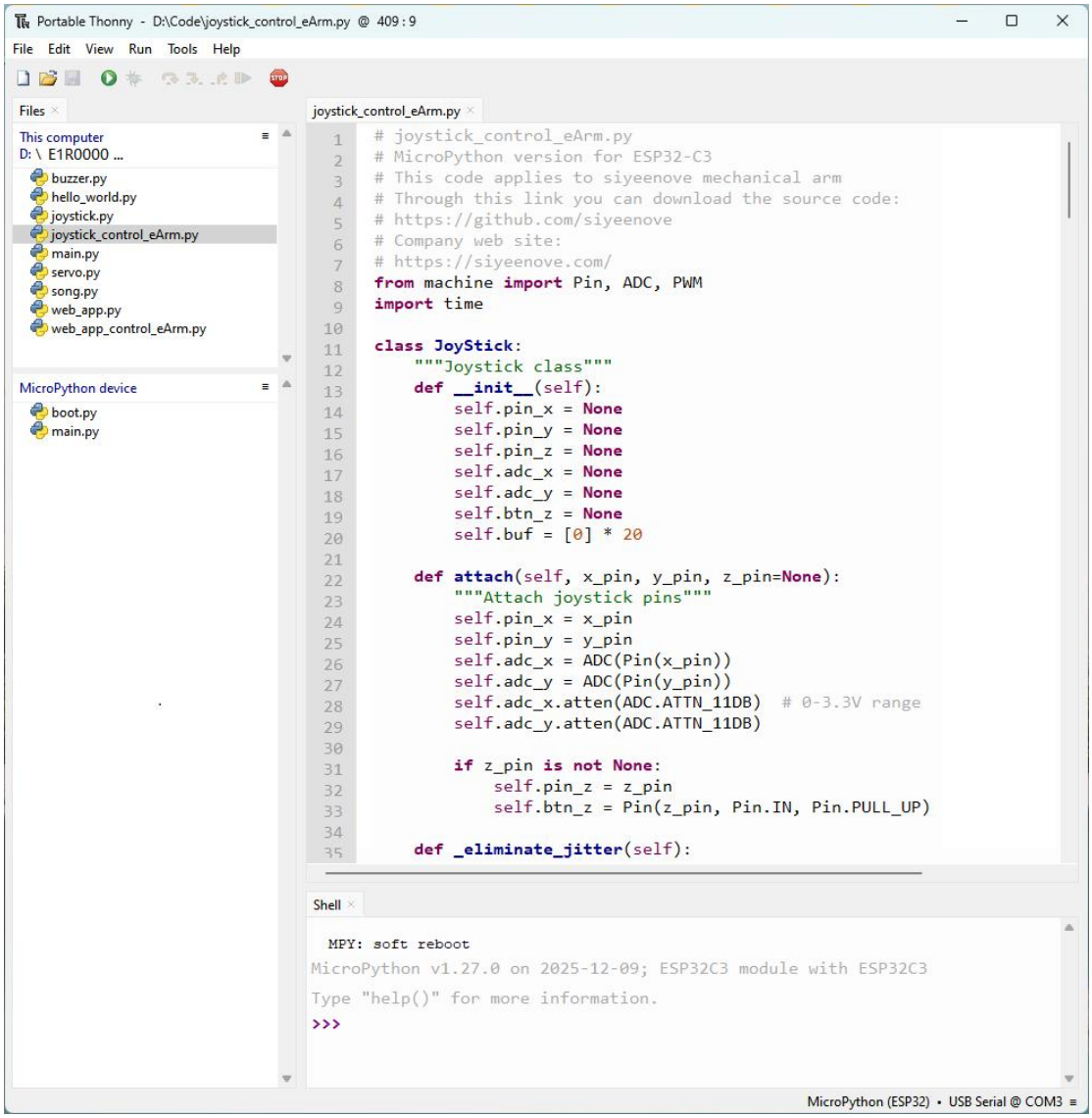
1	Delete the previously uploaded **servo.py** from the MicroPython device, because the provided main.py is designed to call only one module file.
2	If any code is running, please click the **Stop** button.Then right-click on **joystick.py** and select "**Upload to** /"
3	The **joystick.py** file will appear in the MicroPython device file list.
4	Ensure the control board's power switch is turned ON and the battery has sufficient charge.
5	Press the **Reset** button on the esp32 control board to restart it. Push the joystick left, right, up, down, and the shell will print the joystick value
6	To **view** the code: Navigate to the **joystick.py**, then double-click the file to open it in Thonny.



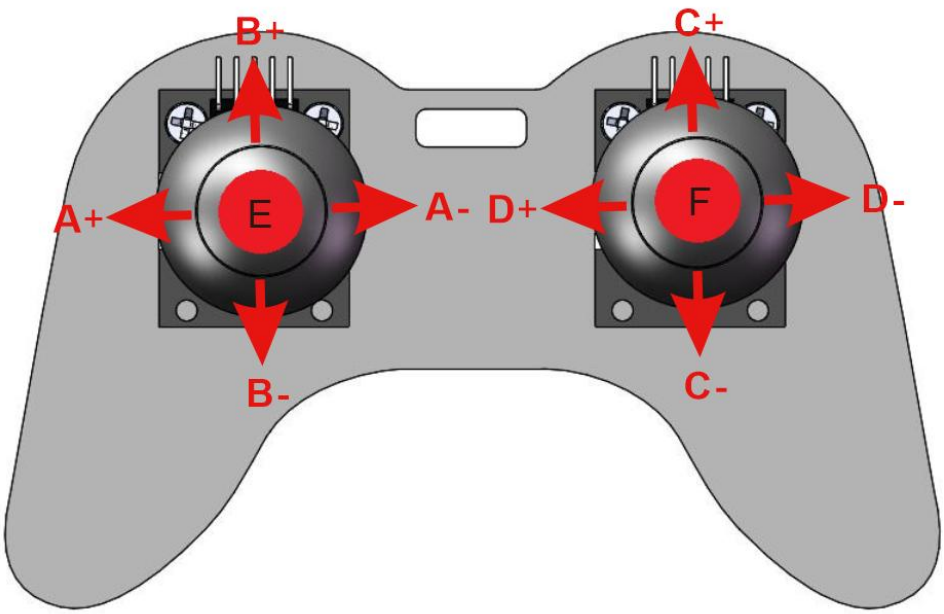


1.8.4 Joystick Control Robot (With Motion Record & Repeat function)

1	Delete the previously uploaded **joystick.py** from the MicroPython device, because the provided main.py is designed to call only one module file.
2	If any code is running, please click the **Stop** button.Then right-click on **joystick_control_eArm.py** and select "**Upload to /**"
3	The **joystick_control_eArm.py** file will appear in the MicroPython device file list.
4	Ensure the control board's power switch is turned ON and the battery has sufficient charge.
5	Press the Reset button on the esp32 control board to restart it. You can push or press the joystick to control the robot arm. **You can even record and replay the robot's movements.**
6	To **view** the code: Navigate to the **joystick_control_eArm.py**, then double-click the file to open it in Thonny.



Joystick Control – Robotic Arm Operation Guide:

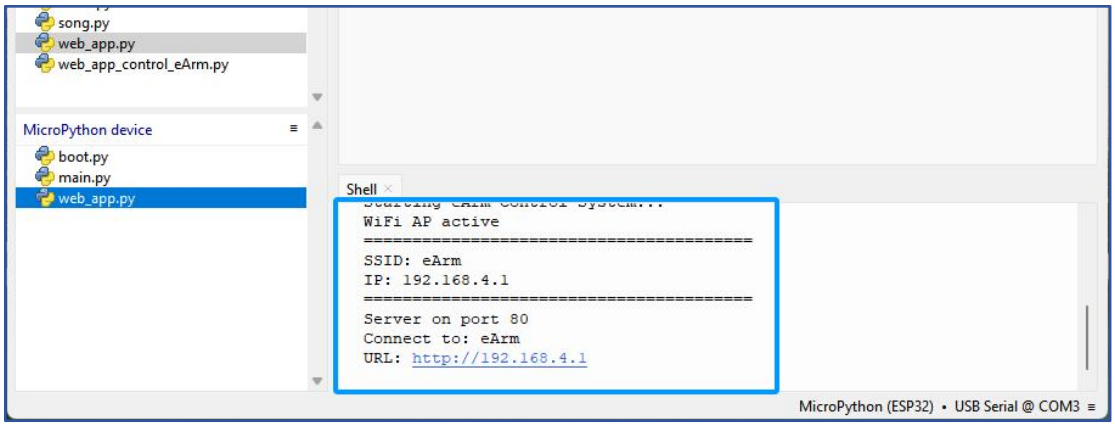


A+: Controls A servo, robotic arm rotates left.	A-: Controls A servo, robotic arm rotates right.
B+： Controls B servo, robotic arm big arm swings forward.	B-： Controls B servo, robotic arm big arm swings backward.
C+: Controls C servo, robotic arm small arm swings forward.	C-： Controls C servo, robotic arm small arm swings backward.
D+: Controls D servo, robotic arm gripper opens (loosens).	D-: Controls D servo, robotic arm gripper closes (tightens).
E： Record Action – Press E button once to record one action (up to 20 actions can be recorded in sequence). Each press triggers a short beep. When the maximum recording capacity is reached, the buzzer emits one long beep.	**F： Playback** — After recording your sequence, press the F button once to start looping the recorded actions continuously. The buzzer will beep once, and the arm will repeat the actions in a loop. To exit the loop, press and hold the F button (long press); the buzzer will emit one long beep to indicate playback has stopped.

1.8.5 Read the Value of the Web APP

1	Delete the previously uploaded **joystick_control_eArm.py** from the MicroPython device, because the provided main.py is designed to call only one module file.
2	If any code is running, please click the **Stop** button.Then right-click on **web_app.py** and select "**Upload to /**"
3	The **web_app.py** file will appear in the MicroPython device file list.
4	Ensure the control board's power switch is turned ON and the battery has sufficient charge.
5	Press the Reset button on the esp32 control board to restart it. You can see the network service startup information of the esp32 in the Shell. Once connected to the ESP32's WiFi, open the web app in a browser. Each button click logs its value in the Shell.
6	To **view** the code: Navigate to the **web_app.py**, then double-click the file to open it in Thonny.

After uploading the code, the shell will print the wifi information.

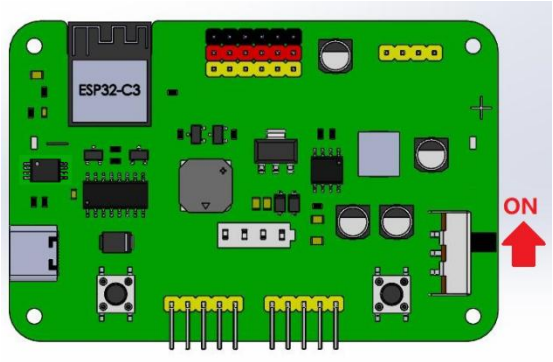


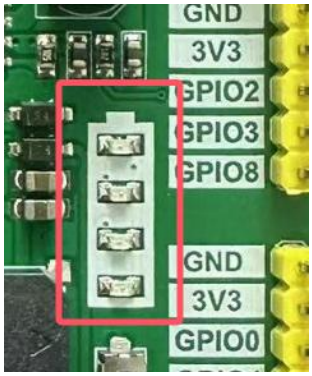
To connect to WiFi, please follow the steps below.

1) Turn the eArm's power switch to the ON position.

■: Make sure the 18650 lithium battery is installed.

■: The battery indicator shows 3 bars or more.





2) Turn on the phone's Wi-Fi and connect to the network named 'eArm'

■: Once connected to the Wi-Fi, you may see a 'Network connection failed' message; just disregard it.

■: Turn off mobile data in your phone settings

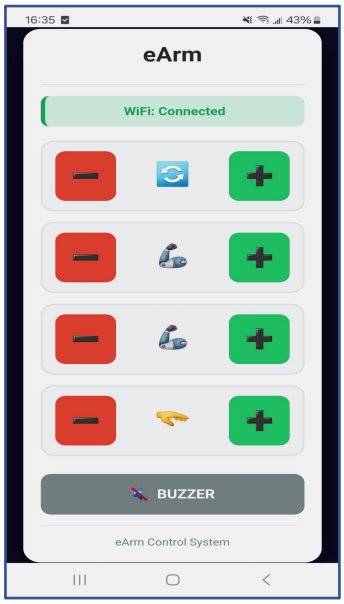
■: This ensures stable connection to the Web App

3) Open the web browser on your phone

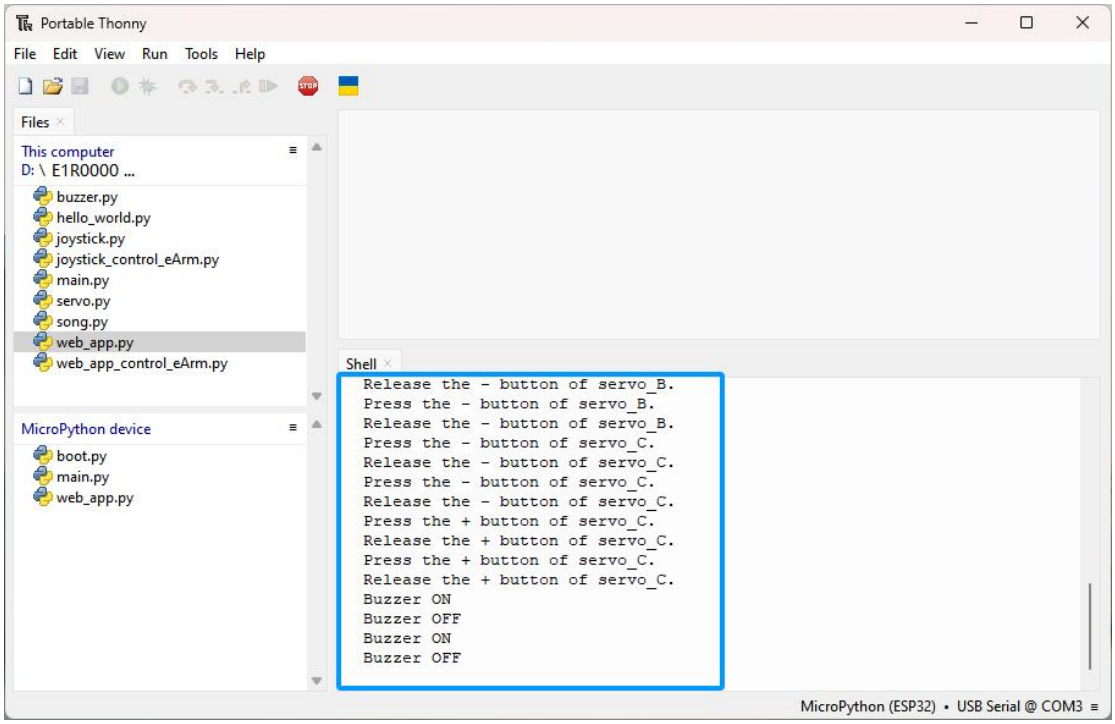
4) Type 192.168.4.1 into the address bar

5) Press Enter to connect

After wifi successful connection, the app control interface will appear in your browser.

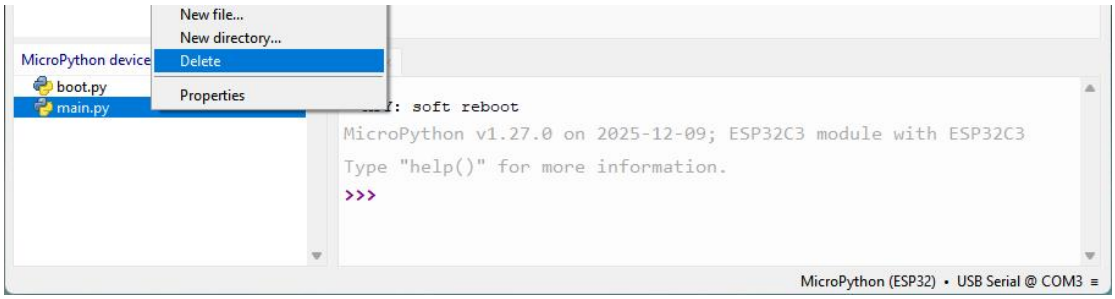


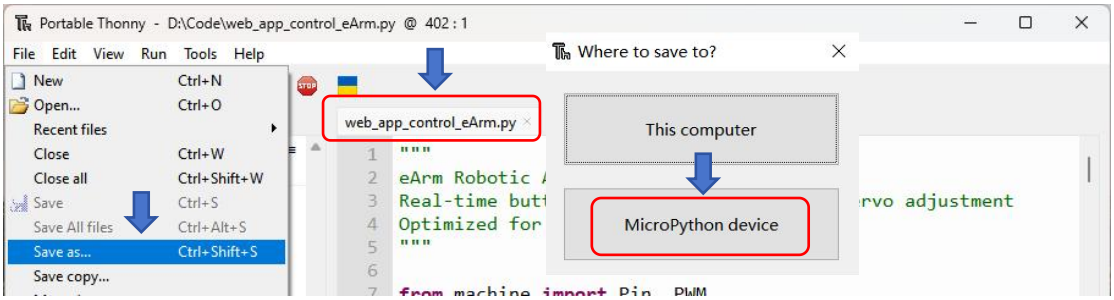
Click the button on the web app and the shell will print the key information.

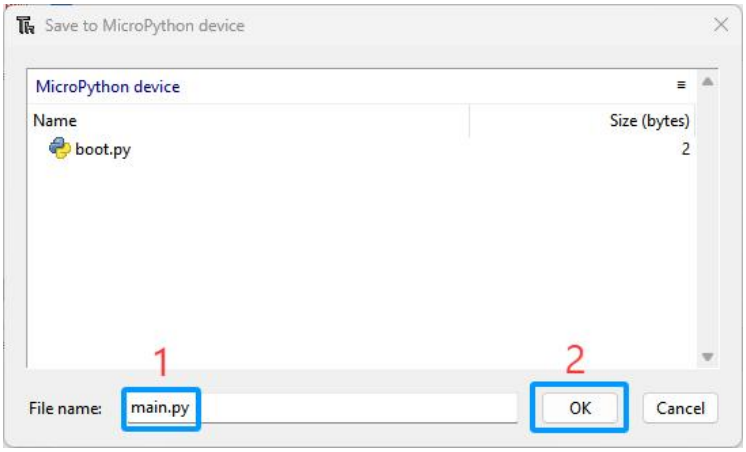


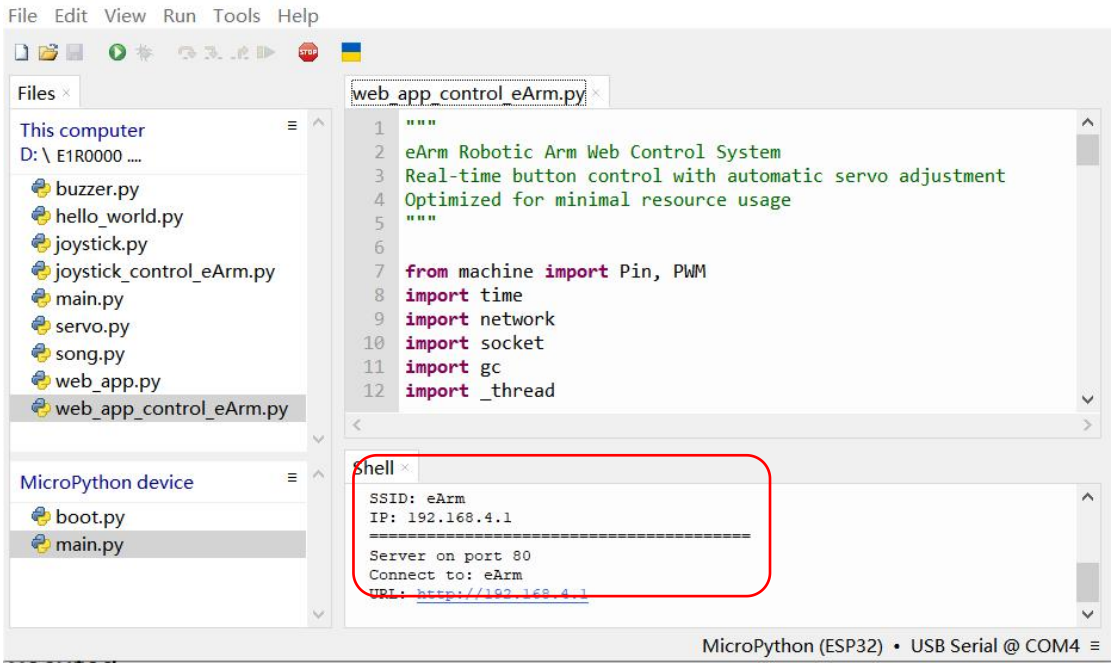
1.8.6 Web App-Controlled Robot Arm

1	Delete the previously uploaded **web_app.py** and **main.py** from the MicroPython device.
2	In this final lesson, directly **save** the code directly **as main.py** and upload to the MicroPython device. Avoid memory overload and ensure smooth operation.
3	Navigate to the **web_app_control_eArm.py**, then double-click the file to open it in Thonny.
4	Click ***File > Save as...*** and select MicroPython device. Name the file **main.py** and click Ok to proceed.
5	Ensure the control board's power switch is turned ON and the battery has sufficient charge.
6	**Press** the Reset button on the esp32 control board to restart it. You can see the network service startup information of the esp32 in the Shell. Once connected to the ESP32's WiFi, you can open the web app in a browser to control the robot arm

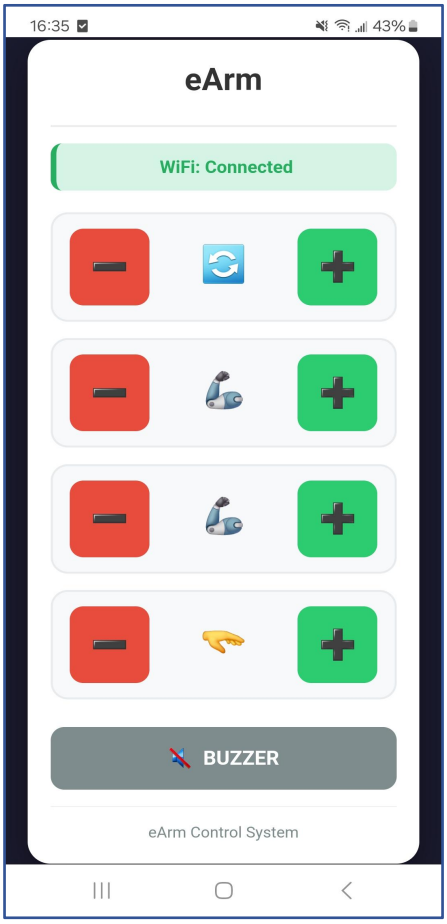


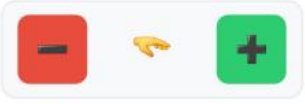






Connect to the ESP32 Wi-Fi using the method from the previous lesson, or click here. Once your device connected to the ESP32's WiFi, you can open the web app in a browser to control the robot arm.



		Robot arm turns left
		Robot arm turns right
		Rear arm raises
		Rear arm lowers
		Front arm raises
		Front arm lowers
		Gripper opens
		Gripper closes
		Buzzer ON
		Buzzer OFF

Important! Critical Gripper Maintenance (Servo Protection)
Continuously gripping an object under load places significant stress on the servo motor (particularly the D-axis driving the gripper), causing it to generate heat. Therefore, we strongly recommend limiting continuous load-bearing gripping to no more than 40 seconds. After completing a gripping operation, release the object immediately by opening the gripper to allow the motor to cool down and rest. Exceeding this duration is the primary cause of motor overheating and burnout.
Our latest optimized firmware/code includes a protective feature. If the gripper servo does not receive a new command within 40 seconds, it will automatically disengage (cease PWM signal transmission) to prevent overheating.
Best Practice: Actively control the gripper. When not manipulating objects, open the gripper or send commands to maintain it in a relaxed state.

2

Burn Firmware

Providing the device's "factory reset program" directly to users in the form of a firmware file (e.g., .bin or .hex), allowing them to flash it into the device without setting up an Thonny IDE, thereby completing the restoration process.

1. Issues with the Traditional Approach

Typically, restoring a device via Thonny requires users to:

- Install the Thonny IDE or related development tools.
- Download the source code (.py file), possibly requiring additional library setups and dependencies.
- Compile and upload it to the device.

This process has a high barrier for non-technical users and may fail due to environment-related issues.

2. Advantages of Providing Firmware Directly

Simplified User Process:

- Provide a pre-compiled firmware file (e.g., firmware.bin), allowing users to flash it with a simple tool (e.g., a serial flasher) in one step.
- No need to install Thonny IDE resolve compilation errors, or manage library conflicts.

Use Cases:

- Restoring a malfunctioning device to factory settings.
- Mass-flashing the same firmware to multiple devices (improves efficiency).

2.1 Burning Tool

Burning Tool is stored in the following folder:

You can also download the latest version Burning Tool from the official website:
https://www.espressif.com/en/support/download/other-tools

Flash Download Tools

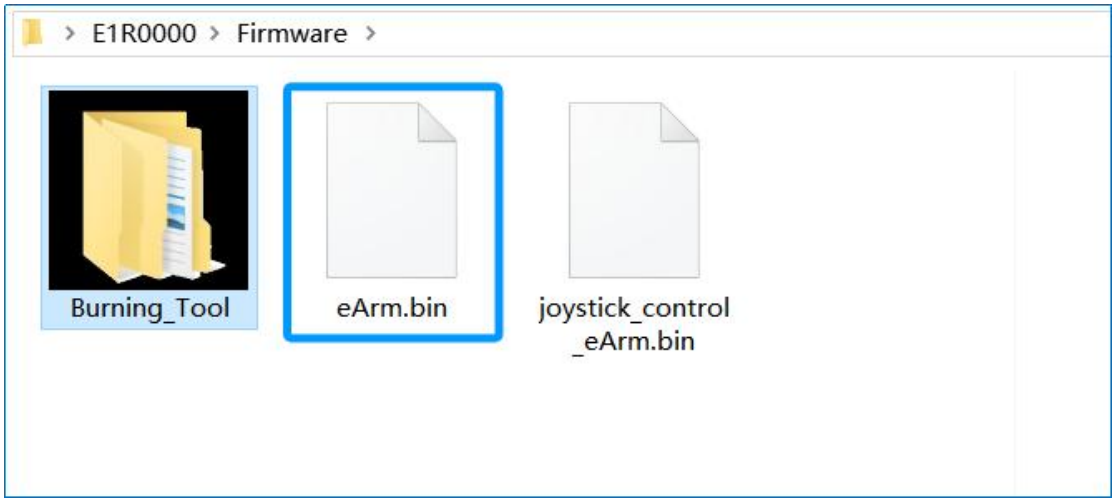
Expand all +

Please refer to Flash Download Tool User Guide to download this tool.

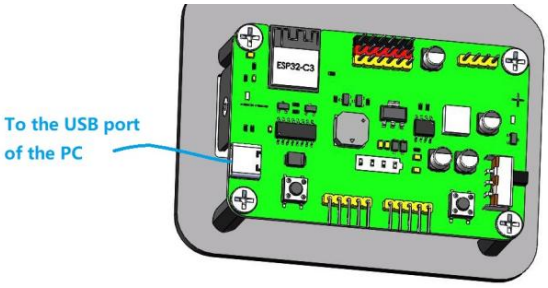
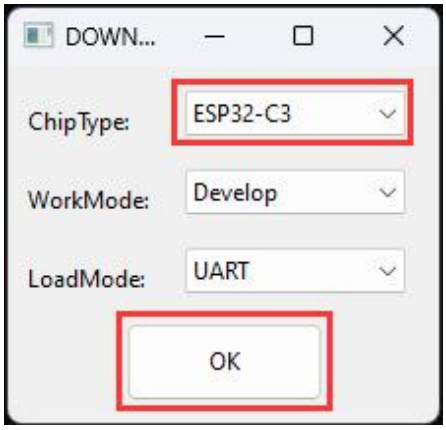
Title	Platform	Version	Release Date	Download
+ Flash Download Tools	HTML	latest	2024.12.18	

2.2 eArm Firmware (Factory Firmware)

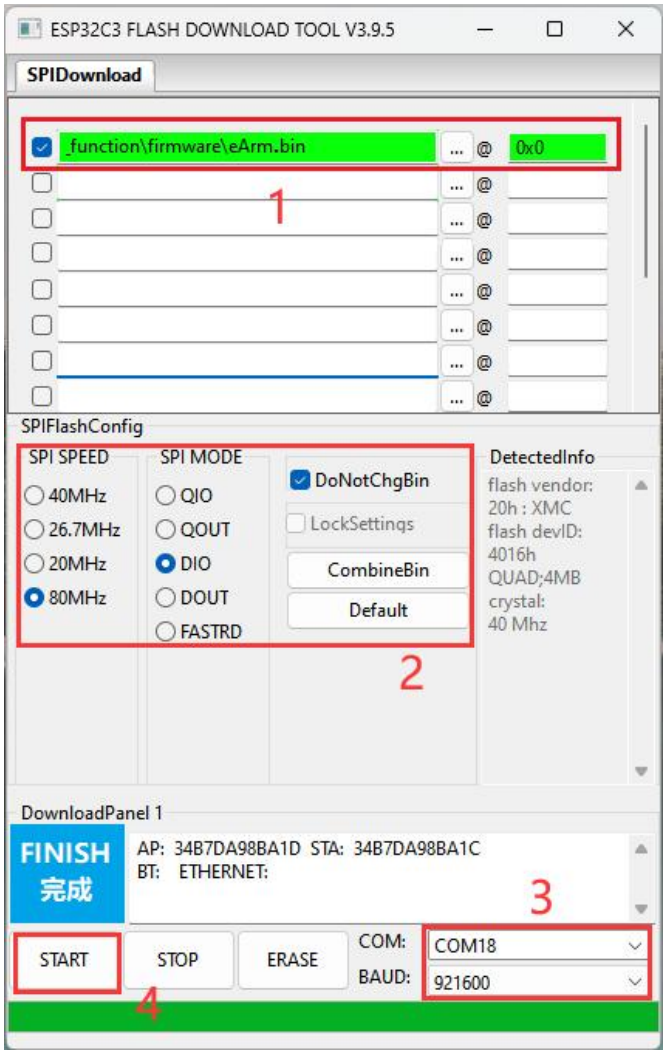
This firmware has the exact same functionality as the factory robotic arm. When the factory functions are lost, this firmware can be used to restore them. It is saved in the following file:



2.2.1 Burning eArm Firmware

Use the USB cable to connect the eArm to PC:	Start the burning tool:
	

Select the firmware to be burned and correctly fill in the burning address.



The CH340 driver must be installed, the battery must be properly connected, and the power switch on the eArm must be turned on; otherwise, the COM port will not be detected.

1： Check one burning option , then click , and select the "**eArm.bin**" firmware file provided in our tutorial. Then, correctly enter the burning address (**0x0**) after "@":

File Name	Burning Address
eArm.bin	0x0

2: Select **80MHz** for SPI SPEED, **DIO** for SPI MODE, and check **DotNotChgBin**.

3： Select the serial port for the control board. Here, it is COM18 (Please select the port based on the actual port assigned to this device by your computer). Set the baud rate to **921600**. If the burning fails, you can lower the baud rate, such as **115200**.

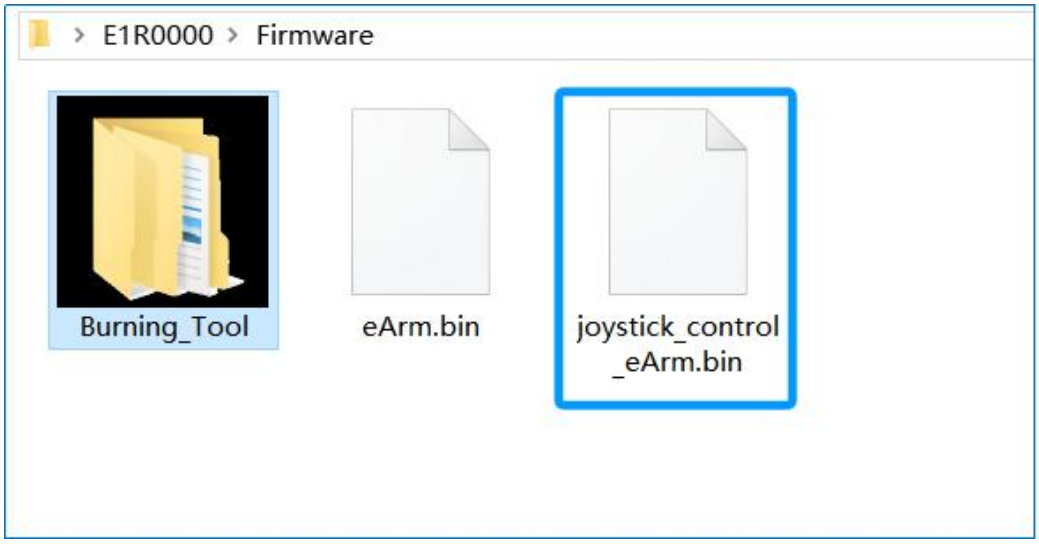
4： Click to start burning the firmware.

After burning is completed, you can follow "**Assembly and User Guide**" to control the robotic arm!

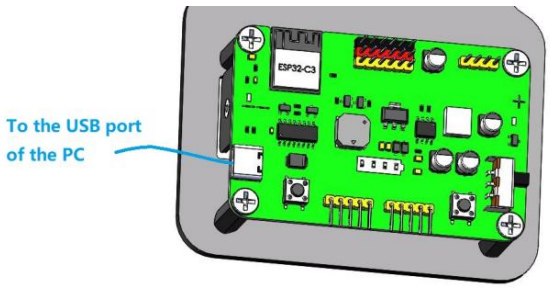
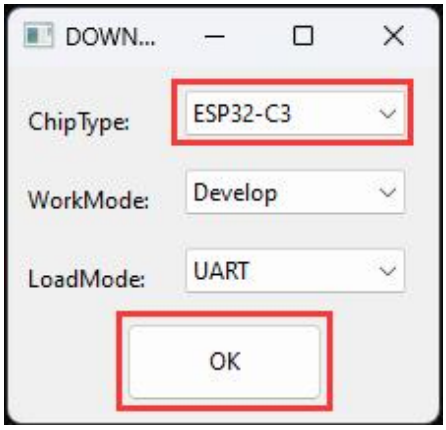
Note: Once the firmware flashing is complete, you need to press the RST (Reset) button or disconnect the USB cable, and then power on.

2.3 Joystick Control Firmware (with Recording Function)

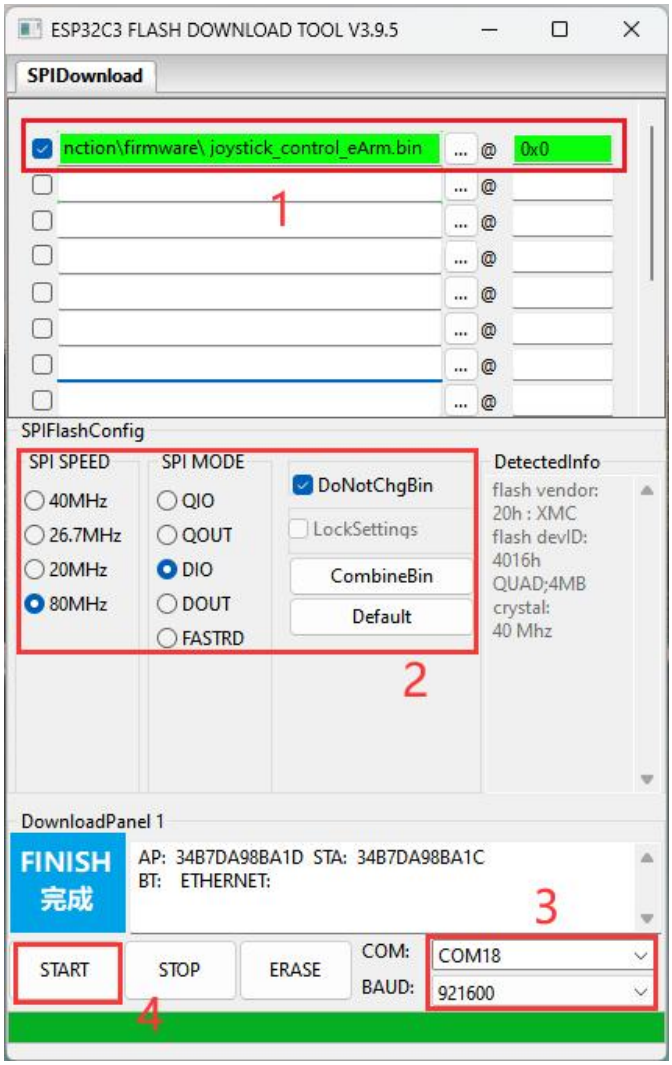
This firmware has the same functions as the code in section 4.8.4, with a recording capacity of 20 actions. Once this firmware is flashed, the joystick control and motion recording/playback features are available without the need to install Thonny. The firmware is saved in the following file:



2.3.1 Burning Joystick Control Firmware

Use the USB cable to connect the eArm to PC:	Start the burning tool:
	

Select the firmware to be burned and correctly fill in the burning address.



The CH340 driver must be installed, the battery must be properly connected, and the power switch on the eArm must be turned on; otherwise, the COM port will not be detected.

1: Check one burning option  , then click  , and select the "joystick_control_eArm.bin" firmware file provided in our tutorial. Then, correctly

enter the burning address (**0x0**) after "@":

File Name	Burning Address
joystick_control_eArm.bin	0x0

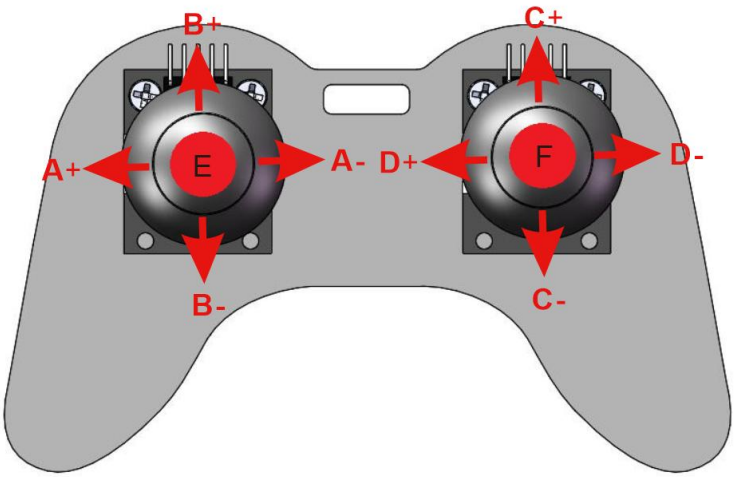
2: Select **80MHz** for SPI SPEED, **DIO** for SPI MODE, and check **DotNotChgBin**.

3: Select the serial port for the control board. Here, it is COM18 (Please select the port based on the actual port assigned to this device by your computer). Set the baud rate to **921600**. If the burning fails, you can lower the baud rate, such as **115200**.

4: Click to start burning the firmware.

Once the firmware flashing is complete, you need to press the RST (Reset) button or disconnect the USB cable, and then power on.

Joystick Control – Robotic Arm Operation Guide:



A+: Controls Aservo, robotic arm rotates left.	A-: Controls Aservo, robotic arm rotates right.
B+ : Controls Bservo, robotic arm big arm swings forward.	B- : Controls Bservo, robotic arm big arm swings backward.
C+: Controls Cservo, robotic arm small arm swings forward.	C-: Controls Cservo, robotic arm small arm swings backward.
D+: Controls Dservo, robotic arm gripper opens (loosens).	D-: Controls Dservo, robotic arm gripper closes (tightens).
E: Record Action – Press E button once to record one action (up to 20 actions can be recorded in sequence). Each press triggers a short beep. When the maximum recording capacity is reached, the buzzer emits one long beep.	**F : Playback** — After recording your sequence, press the F button once to start looping the recorded actions continuously. The buzzer will beep once, and the arm will repeat the actions in a loop. To exit the loop, press and hold the F button (long press); the buzzer will emit one long beep to indicate playback has stopped.

3

Common Issues & Solutions

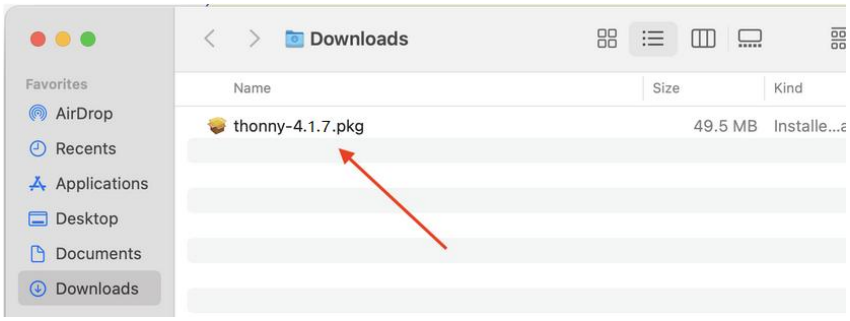
Common Issues & Solutions for ESP32-C3 Programming with Thonny IDE (Windows Version) - Beginner's QA

This QA compiles the **most frequent errors, unexpected behaviors, and operation problems** beginners encounter when programming ESP32-C3 with Thonny IDE on Windows. Each section includes **error symptoms, root causes, step-by-step solutions**, and practical error cases/screenshots description (for easy identification). All operations are beginner-friendly and aligned with MicroPython development workflows.

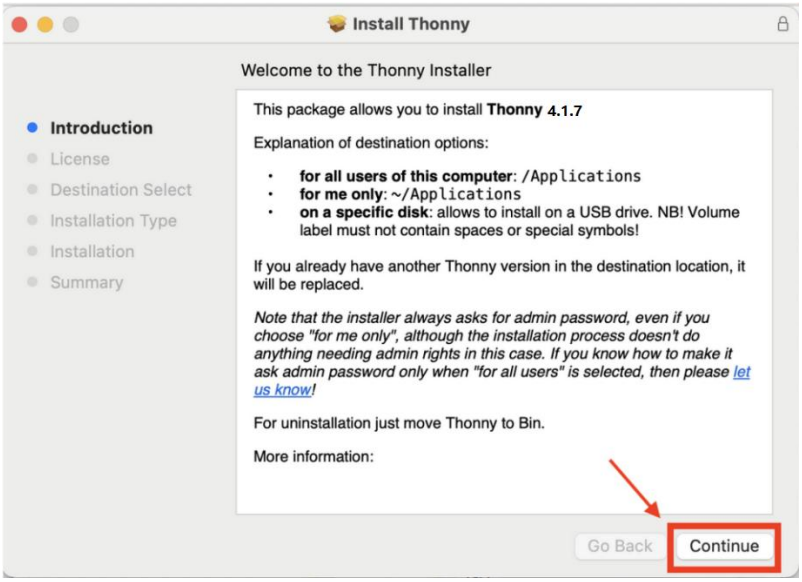
3.1 How to Install Thonny IDE on MacOS?

1) Go to the Thonny office website: https://Thonny.org/
2) Click to download the **Thonny-4.1.7.pkg (42 MB)** installer.

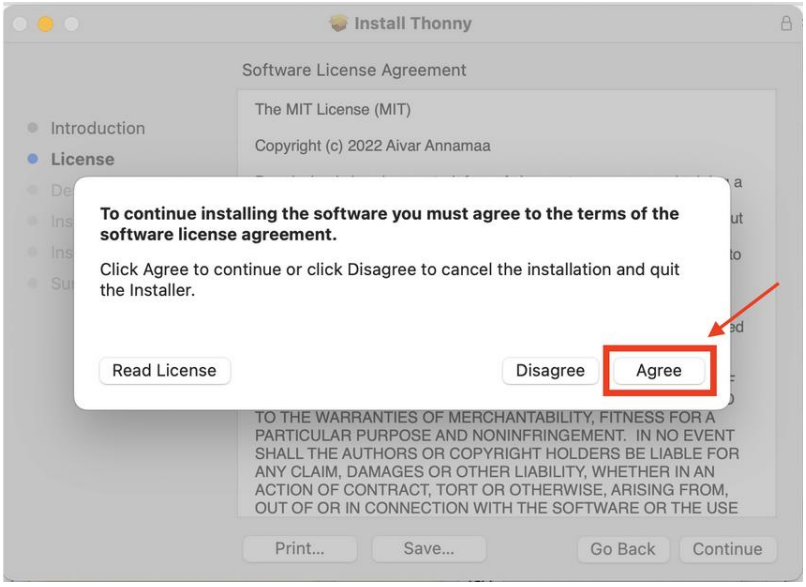
3) Open and Run the Installer and click "Allow"



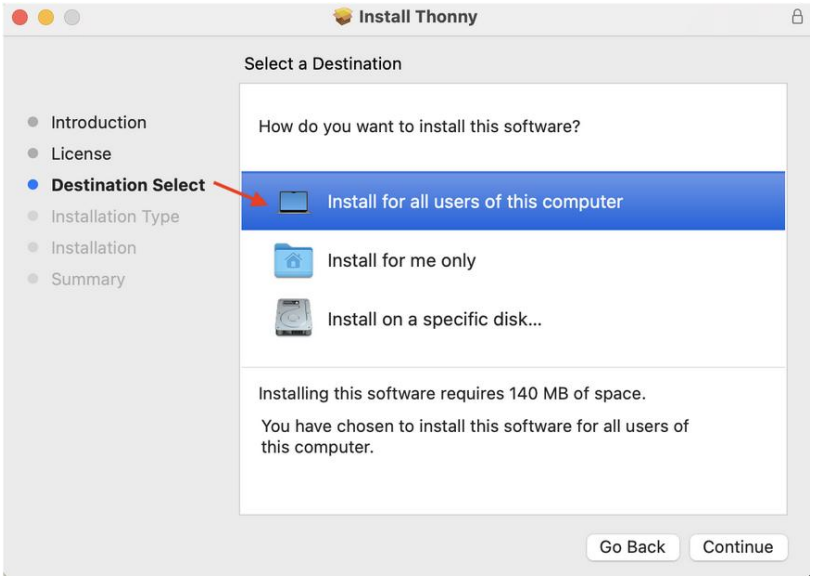
4) Click "Continue" to Confirm the Installation



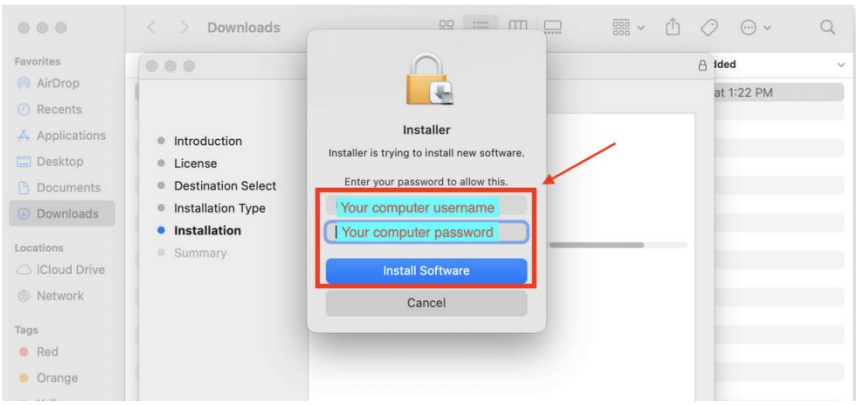
5) Keep clicking “Continue” and click “Agree” to the license agreements



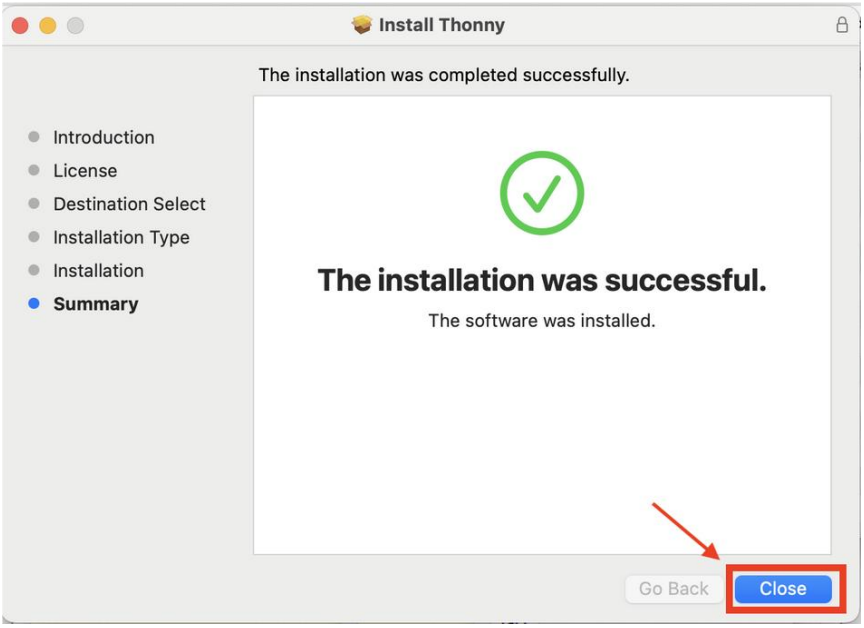
6) “Select a destination” for installation. It is okay to chose the default option.



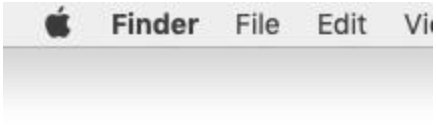
7) Click “Install” and “OK” to allow the installer to access the folders, if necessary. Allow the installation to complete



8) Click “Close”



9) Thonny IDE is now installed and you can double-click to open it:



3.2 Robotic arm doesn't work

☛Whether to turn on the power switch during installation to initialize the angle of the servo.

☛Please check whether the assembly is correct?

☛Please check whether the battery power is sufficient?

☛Please check whether the battery used in accordance with the specifications?

3.3 Port Connection & Device Recognition Errors

Q1: The port menu in the bottom right corner of Thonny does not have the "MicroPython (ESP32)" option.

Error Symptoms

- Thonny IDE bottom-right corner displays **"Local Python 3.x"** only, no **MicroPytho (ESP32)** option.
- When opening **Tools → Options → Interpreter → MicroPython (ESP32)**, there is no "**USB SeriaI @ COMx**" Port selection in the "**Port or WebREPL**" drop-down menu.

Root Causes

1. Missing **CH340 USB-to-Serial driver** (ESP32-C3 development boards use this chip for USB communication, Windows does not have it preinstalled);
2. Loose USB cable (using a charge-only cable instead of a data cable);
3. ESP32-C3 not powered on (USB not plugged in or board power switch off);
4. Damaged USB port (front panel USB ports on desktops are often unstable).

Step-by-Step Solutions

1. **Install the USB-to-Serial driver** (critical step):

- Follow 1.3 Install CH340 USB driver to install

2. **Check the USB cable and port**:
   - Replace with a **data USB cable** (charge-only cables have no data pins and cannot transmit serial data);
   - Plug the USB cable into a **rear panel USB port** (desktop) or a direct laptop USB port (avoid USB hubs);
3. **Verify port in Windows Device Manager**:
   - Press `Win + X` → select **Device Manager** → expand the **Ports (COM & LPT)** section;
   - If ESP32-C3 is recognized, you will see `USB-SERIAL CH340 (COMx)` (x = number like 3,4,5);
   - If a yellow exclamation mark appears on the port, right-click → **Uninstall device** → re-plug the USB cable to reinstall the driver.

Q2: Thonny displays "could not open port 'COMx'" when you click STOP

Error Symptoms

- When you click **STOP** button in Thonny, the Shell will pop up the error "could not open port 'COM3'".
- The port is visible in Device Manager but cannot be used in Thonny.

Root Causes

1. The serial port is occupied by **other software** (e.g., Arduino IDE, Serial Monitor, Putty, VS Code serial plugins);
2. Thonny's previous serial connection was not closed properly (IDE cache issue);
3. Windows background process is holding the serial port.

Step-by-Step Solutions

1. **Close all serial port-related software**:
   - Close Arduino IDE, Serial Monitor, Putty, and any other tools that may use the serial port;

- Check the Windows taskbar for hidden background software (right-click taskbar → Task Manager → end processes related to serial tools).

2. **Restart Thonny IDE:**
   - Close Thonny completely (click X in the top-right corner, ensure no Thonny process in Task Manager);
   - Reopen Thonny and reselect the ESP32-C3 port.
3. **Replug the ESP32-C3 USB cable:**
   - Unplug the USB cable, wait 2 seconds, then plug it back in—this resets the serial port and releases the occupation.

3.4 MicroPython Firmware & Interpreter Configuration Errors

Q1: Thonny prompts "Device is busy or does not respond" when running code

Error Symptoms

- Click the **Run\STOP** button. Shell prompt : **Device is busy or does not respond.**

Root Causes

1. ESP32-C3 has **no MicroPython firmware installed** (brand-new boards usually have no firmware preloaded).
2. Wrong interpreter selected in Thonny (e.g., "MicroPython（ESP8266）" instead of "MicroPython (ESP32)").
3. Corrupted MicroPython firmware on the board (due to incomplete flashing or power failure during firmware installation).

Step-by-Step Solutions

1. **Flash firmware in Thonny:**
   i. Refer to Section 4.5 to re-flash the firmware(do not unplug the

USB cable during flashing.

ii. After completion, click **Close** and restart the ESP32-C3 (press RESET button).

2. **Verify firmware installation**:

   - Open Thonny's Shell terminal (**View → Shell**);
   - Press the ESP32-C3's RESET button—you will see the MicroPython version information and the `>>>` prompt (success).

Q2: Thonny shows "Wrong board model" during firmware installation

Error Symptoms

- When installing MicroPython firmware, a pop-up error: **"Wrong board model selected. The firmware is not compatible with ESP32-C3"**;
- Firmware flashing fails with a "Verification error" or "Write error".

Error Case Screenshot Description

Firmware installation progress bar stops at 10-50%, and a red warning pops up: "Firmware for ESP32 (generic) is not compatible with this device. Please select ESP32-C3 as the board model".

Root Cause

Selected the wrong **board model** in the firmware installation page (e.g., "ESP32 (generic)" instead of "ESP32-C3")—ESP32-C3 uses a RISC-V core, and generic ESP32 firmware (Xtensa core) is incompatible.

Step-by-Step Solution

1. Refer to Section 4.5 to re-flash the firmware(do not unplug the USB cable during flashing.

3.5 Runtime Errors (Code Execution on ESP32)

Q1: Code shows "ImportError: no module named 'xxx'" when running on ESP32-C3

Error Symptoms

- Clicking Run or restarting the ESP32-C3 shows a red error in the REPL terminal: **"ImportError: No module named 'led'"**.
- The main code (e.g., `main.py`) fails to execute because it cannot import a custom module.

Error Case Screenshot Description

Shell displays:

```
Plain Text
>>> import led
ImportError: No module named 'led'
>>> main.py:1: ImportError: No module named 'sensor'
```

Root Causes

Case 1: Custom module (e.g., led.py, sensor.py) → Not uploaded to ESP32-C3

- The imported module file is only on the Windows local computer, not in the ESP32-C3's file system.

Case 2: Custom module → Filename case mismatch

- MicroPython is **case-sensitive** for filenames (e.g., `Led.py` ≠ `led.py`, `SENSOR.py` ≠ `sensor.py`).

Step-by-Step Solutions

1. **For custom modules: Upload the module file to ESP32-C3**
    - In Thonny, open the missing module file (e.g., `led.py`);
    - Click **File → Upload to/** to upload it to the ESP32-C3;
    - Verify the file is visible in the Thonny File panel (under the MicroPython device);

- Re-run the main code or restart the board.

2. **For custom modules: Fix filename case mismatch**
   - Ensure the **import filename matches the actual filename exactly** (case and spelling);

     Example: If the file is `LedBlink.py`, the import must be `import LedBlink` (not `import ledblink`);
   - Rename the file to **all lowercase** (recommended for beginners) to avoid case errors (e.g., `ledblink.py`).

Q2: Code shows "OSError: [Errno 2] ENOENT" when running on ESP32

Error Symptoms

- Shell terminal displays **"OSError: [Errno 2] ENOENT"** when executing code;
- This error usually occurs with file operations (e.g., opening a file) or module imports.

Error Case Screenshot Description

```
Plain Text
>>> f = open("data.txt", "r")
OSError: [Errno 2] ENOENT
>>> from led import run
OSError: [Errno 2] ENOENT
```

Root Cause

ENOENT = Error NO ENTity → The file/module the code is trying to access **does not exist** on the ESP32's file system (the most common error for beginners).

- For file operations: The target file (e.g., `data.txt`) is not uploaded to the board;
- For imports: The module file (e.g., `led.py`) is missing or the path is wrong.

Step-by-Step Solutions

1. **Verify the file/module exists on the ESP32-C3**:
    - Check the Thonny File panel to confirm the file (e.g., `data.txt`, `led.py`) is present under the MicroPython device;
    - If missing, upload the file to the board.
2. **Fix the file path in code**:
    - Use **only the filename** for root directory files (no absolute/relative paths like `/flash/led.py` or `./led.py`);

    Correct: `import led` / `f = open("data.txt", "r")`;

    Incorrect: `import flash.led` / `f = open("./data.txt", "r")`.
3. **Check for typos in the filename**:
    - Ensure the filename in the code matches the actual filename (no extra spaces, wrong letters);

    Example: `data.txt` ≠ `data .txt`, `led.py` ≠ `ledd.py`.

Q3: Code runs in Thonny (Run button) but not after ESP32 restart

Error Symptoms

- Clicking the **Run** button in Thonny makes the ESP32's buzzer sounds normally;
- After restarting the board (unplug USB/repress RESET), the buzzer stops emits sound (no code runs automatically).

Error Case Screenshot Description

Shell terminal shows the code executing successfully when clicking Run: `>>> # buzzer...`; after restart, the terminal has only the MicroPython version prompt and no code execution.

Root Causes

1. The code is **not saved as `main.py`** on the ESP32-C3 (MicroPython runs `main.py` automatically on startup);
2. `main.py` is saved on the **Windows local computer**, not uploaded to the board;

3. main.py has **syntax errors** (causes automatic execution to fail on startup).

Step-by-Step Solutions

1. **Ensure the entry code is saved as main.py and uploaded** (core fix):
   - If using single-file code: Save the code as main.py (**File → Save As**) → Upload to the ESP32(**File → Upload to**);
   - If using modular code: Upload all custom modules (e.g., buzzer.py) → Create a minimal main.py (e.g., import buzzer; buzzer.run()) → Upload main.py to the board.
2. **Verify main.py is in the ESP32 root directory**:
   - Check the Thonny File panel to confirm main.py is visible under the MicroPython device (no subfolders);
   - If missing, re-upload main.py.
3. **Check main.py for syntax errors**:
   - In Thonny, open the main.py file on the board (double-click in the File panel);
   - Look for **red underlines** (Thonny's syntax check) → fix errors (e.g., missing colons :, indentation errors, wrong pin numbers);
   - Re-upload the corrected main.py and restart the board.
4. **Test automatic execution**:
   - After uploading the correct main.py, restart the ESP32 → the code runs automatically (buzzer) with no need to click Thonny's Run button.

3.6 File System & Device Display Abnormalities

Q1: The Thonny file panel doesn't shows "MicroPython Device" ?

Error Case Screenshot Description

Thonny File panel displays the MicroPython device entry, but the right pane has red text: "Could not list files on device. Check connection and try again."

Root Causes

1. This phenomenon occurs every time the computer's USB port is connected to the control board, because Thonny lacks the automatic detection feature for ESP32 device connection.
2. Serial port communication is unstable (loose USB cable, bad USB port).

Step-by-Step Solutions

1. **Making Thonny recognize the ESP32 device:**
    - Click the **STOP** button, or reselect "**MicroPython (ESP32) ▪ USB Serial @ COMx**" in the bottom right corner of Thonny.
2. **Stabilize the serial connection:**
    - Replace the USB data cable → plug into a rear desktop USB port (avoid hubs);
    - Ensure the USB cable is not loose (no intermittent LED blinking on the board).

3.7 Serial Port Occupation & Connection Drop Issues

Q1: ESP32-C3 disconnects randomly from Thonny (COM port disappears)

Error Case Screenshot Description

Thonny bottom-right corner suddenly switches from "MicroPython on COM3" to "Local Python 3.x"; Device Manager's Ports section no longer shows the USB-SERIAL CH340 port.

Root Causes

1. **USB power supply instability** (laptop/USB hub provides insufficient power to the ESP32);
2. Damaged USB-to-Serial chip on the ESP32 board (hardware issue);
3. Windows USB driver conflict (outdated CH340 driver);

Step-by-Step Solutions

1. **Improve USB power supply** (most common fix):
    - For laptops: Plug the USB cable into a **powered USB hub** (or connect the laptop to a power adapter);
    - For desktops: Always use **rear panel USB ports** (they provide more stable power than front panels);
    - Avoid using unpowered USB hubs (they cannot supply enough current for the ESP32-C3).

4
Contact Us

If you couldn't find a solution above, please contact our support team.

To help us assist you quickly, please have the following information ready:

Your order number.
Product model (e.g., Robot Arm Kit E1R0000) and software version
A detailed description of the issue or your question.
Steps you have already tried.
Any relevant error message screenshots, photos, or code snippets.

Other Inquiries?

Please feel free to reach out for:

Tutorial Errors & Feedback: Help us make our documentation better.
Product Ideas & Suggestions: We'd love to hear your great ideas.
Partnerships & Collaboration: Interested in working together? Let's talk.
Discounts & Promotions: Inquire about educational or bulk pricing.
Anything Else: For all other non-technical questions.

 support@siyeenove.com

 https://siyeenove.com